



Intro to Cryptography

UofTCTF Seminars 2026



Announcements

- We will be presenting at Deerhacks come find us...
- 2 roles we are hiring for:
 - VP finance
 - VP technology
 - view roles on sop + email us if youre interested
- Maybe we play some wargames this weekend?

Outline

- Cryptographic Theory
- Encodings
- Symmetric Cryptography (Block Ciphers, Stream Ciphers)
- Asymmetric Cryptography (RSA, Key Exchange, ECC)
- Pseudo-Random Number Generators
- Hashing & MAC
- Key Exchange Functions
- Public Key Infrastructure
- Digital Signatures

Cryptographic Theory

Messages & Keys

Message:

A series of bytes containing information to be read



```
0x77 0x28 0xf8 0x29 0x21
0x99 0x82 0xaf 0x2e 0x3b
0x3f 0x22 0x01 0x36 0x99
```

Key:

A smaller series of bytes used to:

- Encrypt a message
- Decrypt a message



```
0x21 0x33 0x2e 0xf8
```

Encryption/Decryption

Ciphertext: The result of a message going through a cipher

Encryption: Plaintext -> Ciphertext via cipher



Decryption: Ciphertext -> Original plaintext via cipher



Cryptography Goals

The goal of cryptography is to establish cryptosystems that ensure:

- Confidentiality
- Integrity
- Authenticity

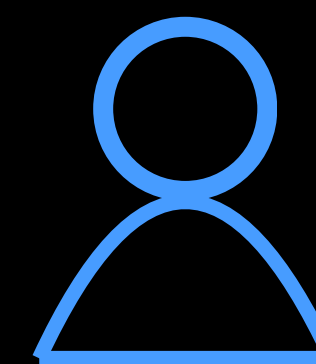
of all messages transmitted.

Confidentiality/Secrecy

The ability to communicate in the presence of external adversarial behavior without leaking sensitive information



Alice



Bob



Eve

Integrity

The ability for data to be checked and validated
against tampering and corruption.

```
10011100001101101010101011110  
11010111000100000101110000001  
11000001000000110000001100111  
11111001000111111001100010111  
11100111110100010101010101101  
11011000010111110010100101011  
11100000111010010010100001011
```

Message

SHA256: 17227448b294bc174e4fb6
79b3060952d095afe36b87
4dc3c4294c3fa5144c9e

```
10011100001101101010101011110  
11010111000100000101110000001  
11000001000000110000001100111  
11111001000111111001100010111  
11100111110100010101010101101  
11011000010111110010100101011  
11100000111010010010100001011
```

Modified Message

SHA256: 7340b3691533df4b1b834f
7cc6894c7bf79b689c9796
79207cb93f6c3ec1d004

SHA256 checksums can be used to check integrity

Authenticity

The ability to determine who wrote a message



```
1001110000110110101010101111  
11010111000100000101110000001  
11000001000000110000001100111  
11111001000111111001100010111  
11100111110100010101010101101  
11011000010111110010100101011  
11100000111010010010100001011
```

Message

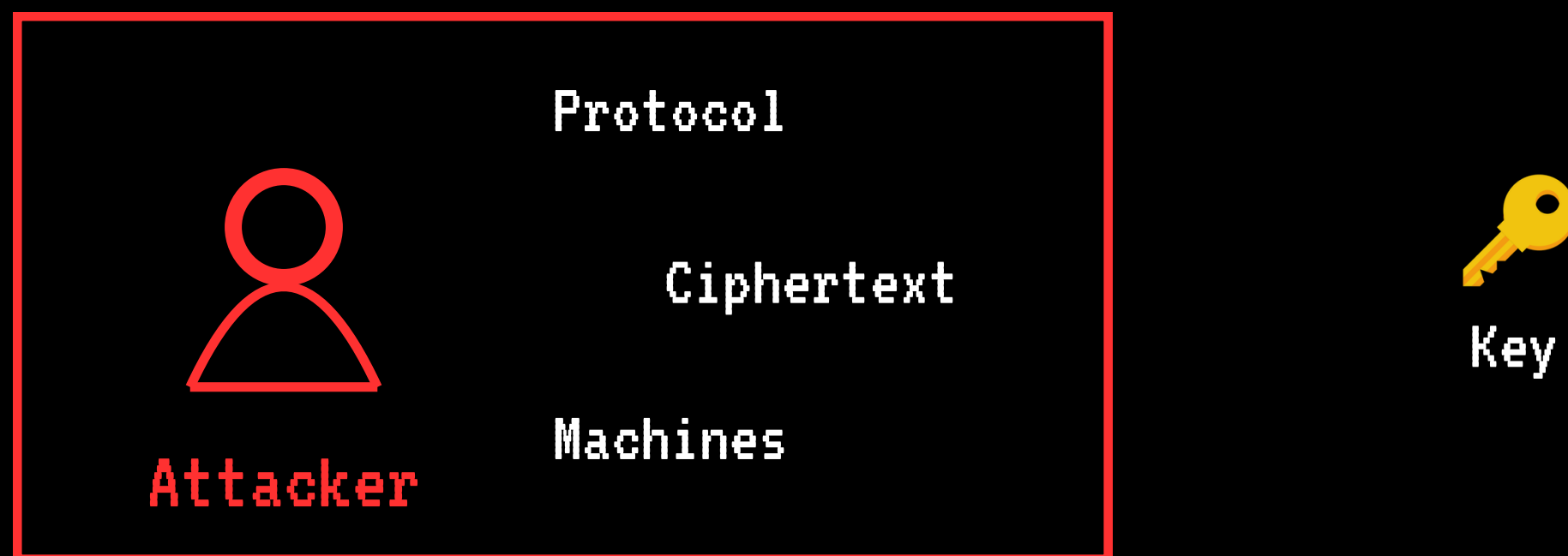
Digital Signature : From Alice

Authenticity protocols often create wrappers around
original message to help with integrity

Good Cryptosystems

Kerckhoff's Principle: A system should remain secure even if everything about the system excluding the key is known.

Shannon's Maxim: The enemy knows the system



Old cryptosystems relied on 'security through obscurity' by keeping the protocols secret, this does not scale.

Games/Attack-Models

We build cryptosystems based off the threat models we are concerned about:

- Cipher Only Attack (COA)
- Known Plaintext Attack (KPA)
- Chosen Plaintext Attack (CPA)
- Chosen Ciphertext Attack (CCA)
- Chosen Plaintext Ciphertext Attack (CPCA)

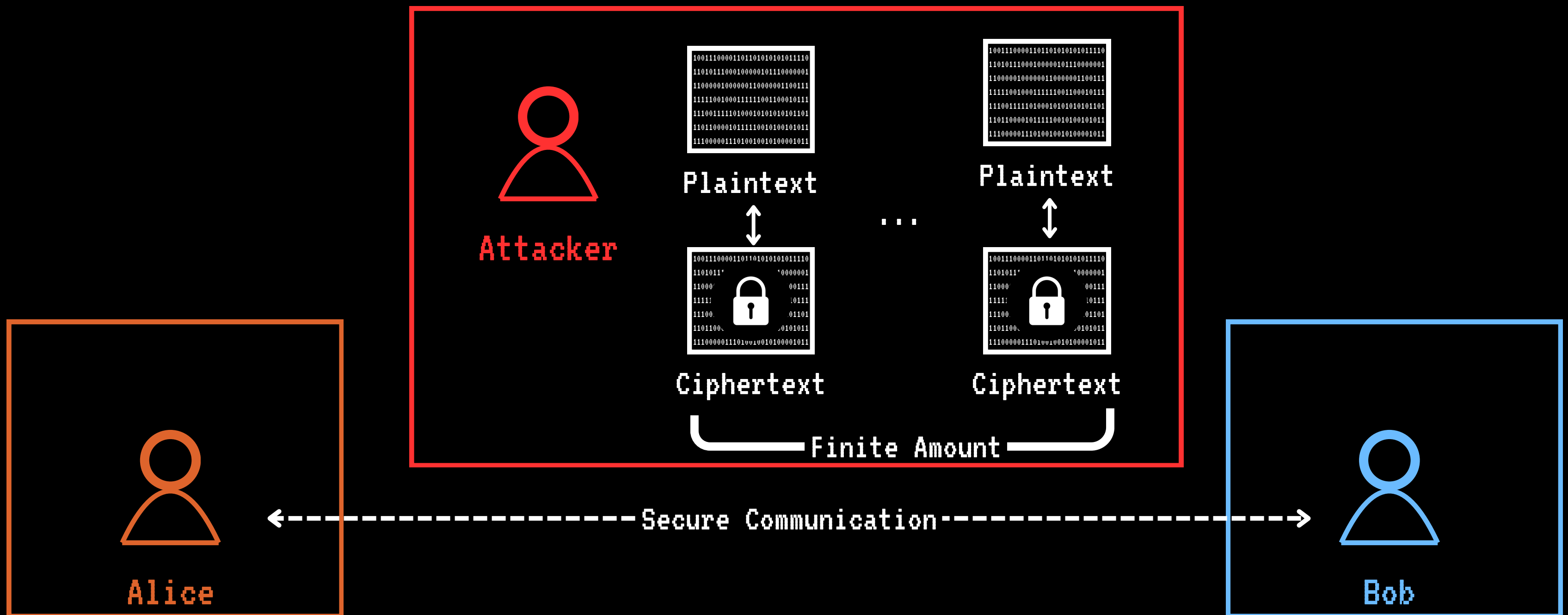
Cipher Only Attack (COA)

Scenario where attacker only knows some ciphertext



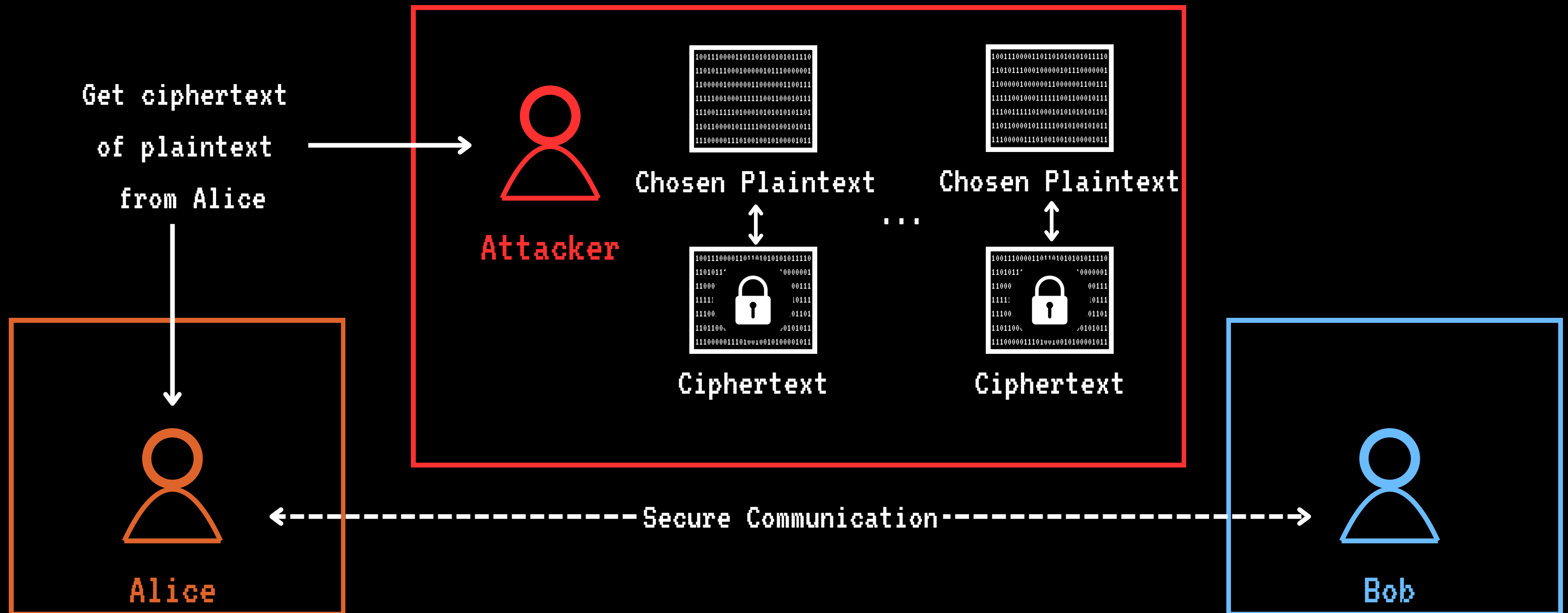
Known Plaintext Attack (KPA)

Scenario where attacker only knows some
ciphertext $\leftrightarrow$ plaintext pairs



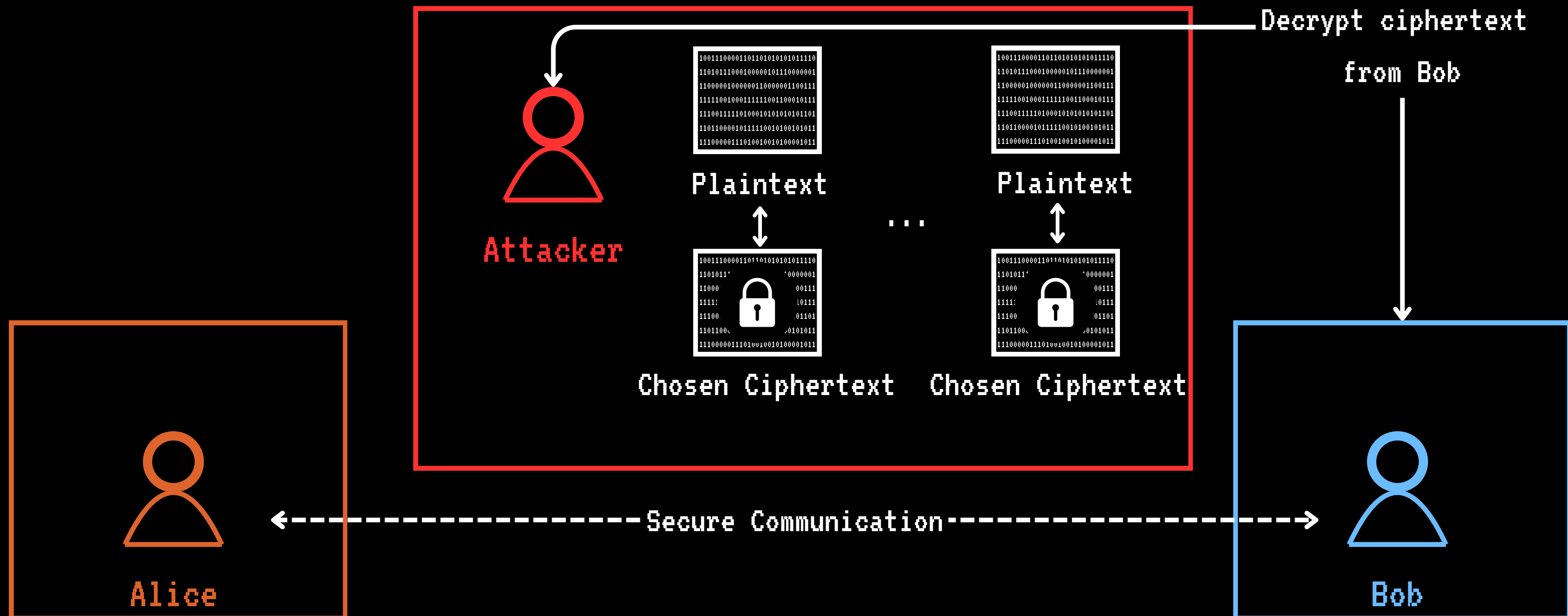
Chosen Plaintext Attack (CPA)

Scenario where attacker can choose which plaintexts to get ciphertext of



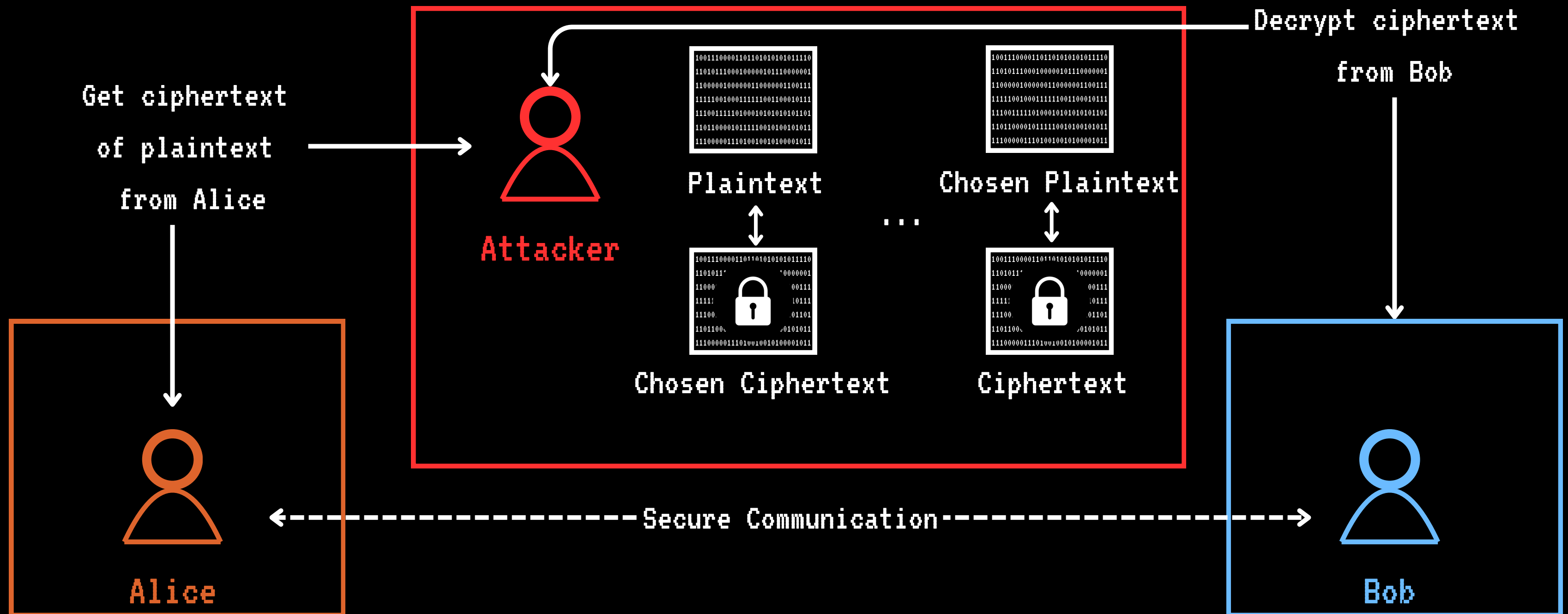
Chosen Ciphertext Attack (CCA)

Scenario where attacker can choose to
decrypt given ciphertexts



Chosen Cipher/Plaintext Attack (CPCA)

Scenario where attacker can choose to decrypt
and encrypt given ciphertexts/plaintexts



Encodings

Encodings

The process of converting information or data
into a specific format.

Example: ASCII

1100001	→	a
1010101	→	U

Base64

Really Cool Text $\longleftrightarrow$ UmVhbGx5IENvb2wgVGV4dA==

"Man"



010011010110000101101110



010011 010110 000101 101110



TW Fu

"Hi"



0100100001101001xx



010010 000110 1001xx



SGk=

Base64 Alphabet

Uppercase Letters		Lowercase Letters		Digits		Symbols	
Index	Character	Index	Character	Index	Character	Index	Character
0	A	26	a	52	0	62	+
1	B	27	b	53	1	63	/
2	C	28	c	54	2		
3	D	29	d	55	3		
4	E	30	e	56	4		
5	F	31	f	57	5		
6	G	32	g	58	6		
7	H	33	h	59	7		
8	I	34	i	60	8		
9	J	35	j	61	9		
10	K	36	k				
11	L	37	l				
12	M	38	m				
13	N	39	n				
14	O	40	o				
15	P	41	p				
16	Q	42	q				
17	R	43	r				
18	S	44	s				
19	T	45	t				
20	U	46	u				
21	V	47	v				
22	W	48	w				
23	X	49	x				
24	Y	50	y				
25	Z	51	z				

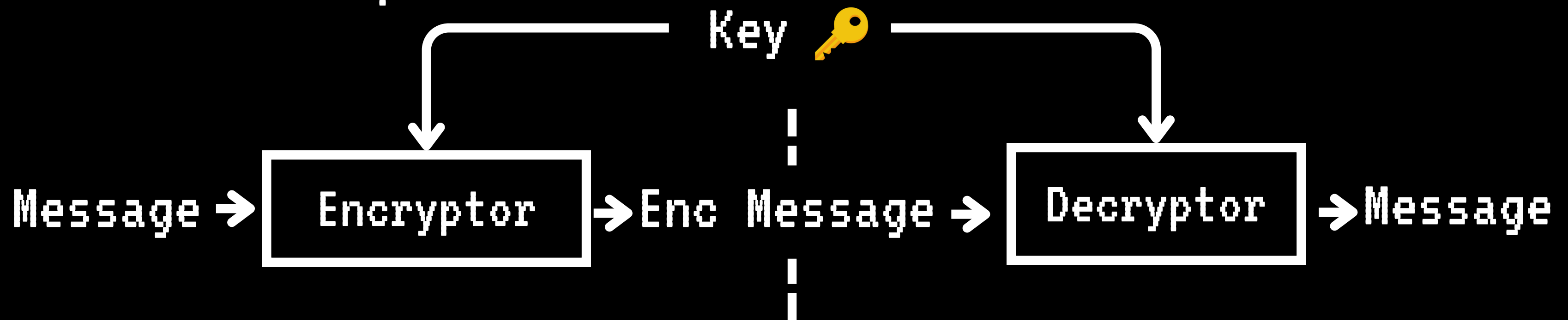
Ciphers

Symmetric-Key Cryptography

Using the same key to encode and decode a message

Symmetric Encrypt/Decrypt Algorithms:

- Block Ciphers
- Stream Ciphers



XOR Review

$$0 \oplus 0 = 0$$

$$0 \oplus 1 = 1$$

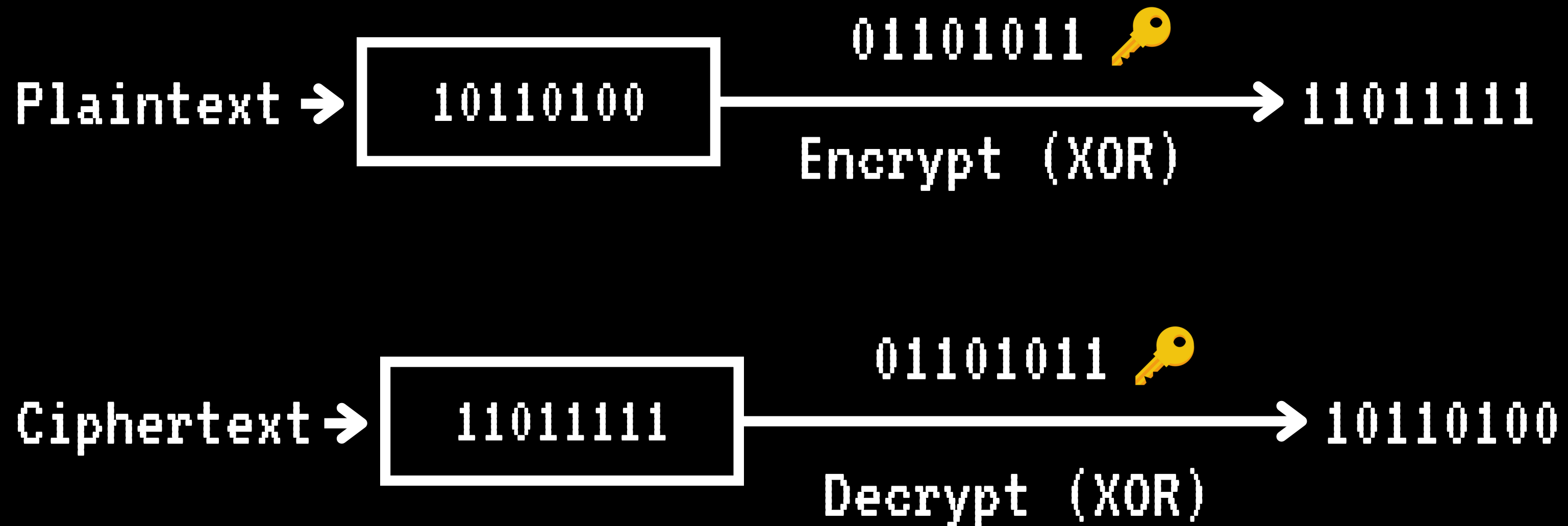
$$1 \oplus 0 = 1$$

$$1 \oplus 1 = 0$$

XOR (exclusive-or) has a
useful property:

$$x \oplus y \oplus y = x$$

One-time Pad (OTP)



Uses a different key every message

One-time Pad (OTP)

OTP has to use a different key every message:

$$\begin{array}{l} C = M \oplus K \\ C' = M' \oplus K' \end{array} \quad \longrightarrow \quad C \oplus C' = M \oplus M'$$

Impractical in the Real World

Block Ciphers

Algorithm that encrypts “blocks” of fixed length

Plaintext



Key



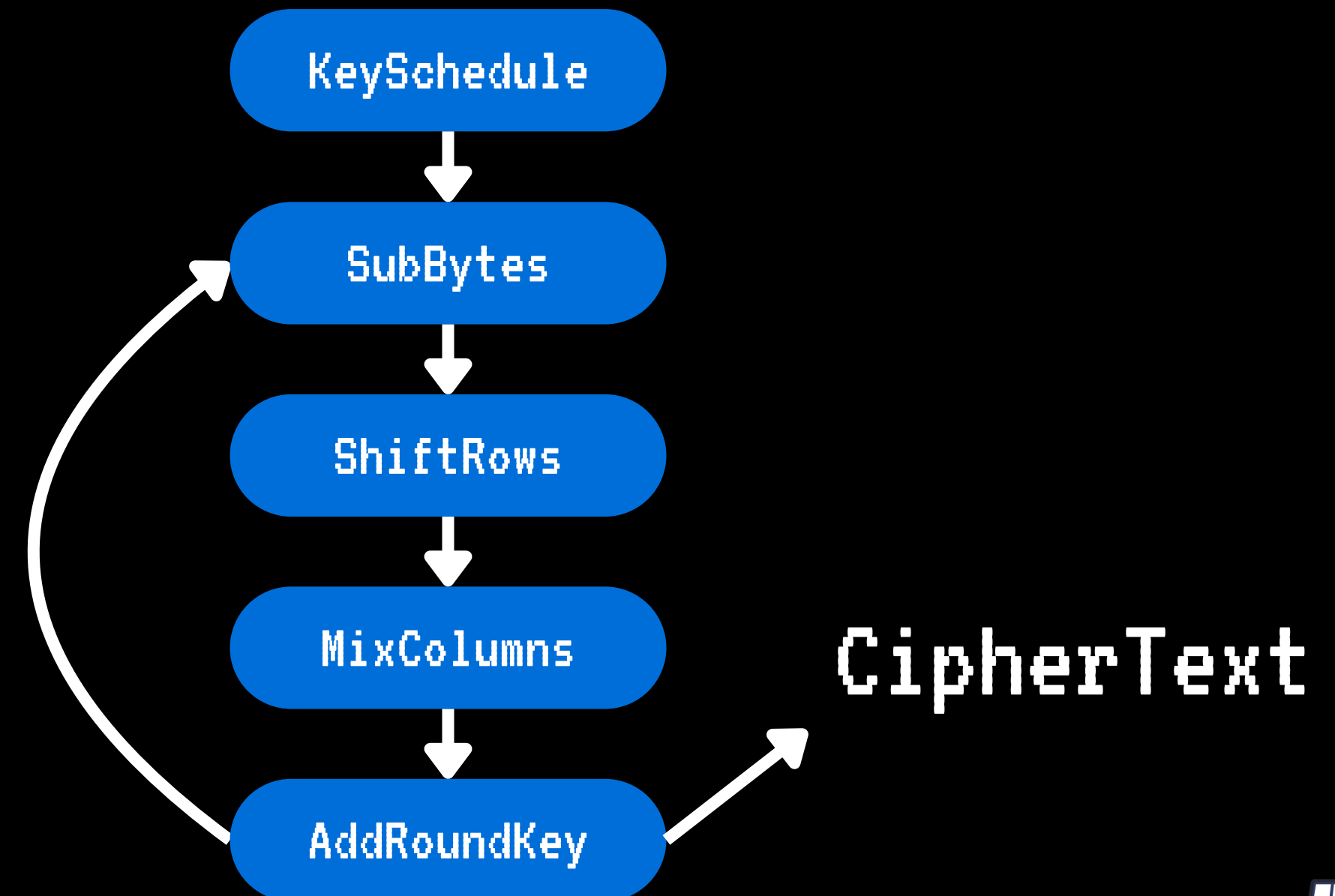
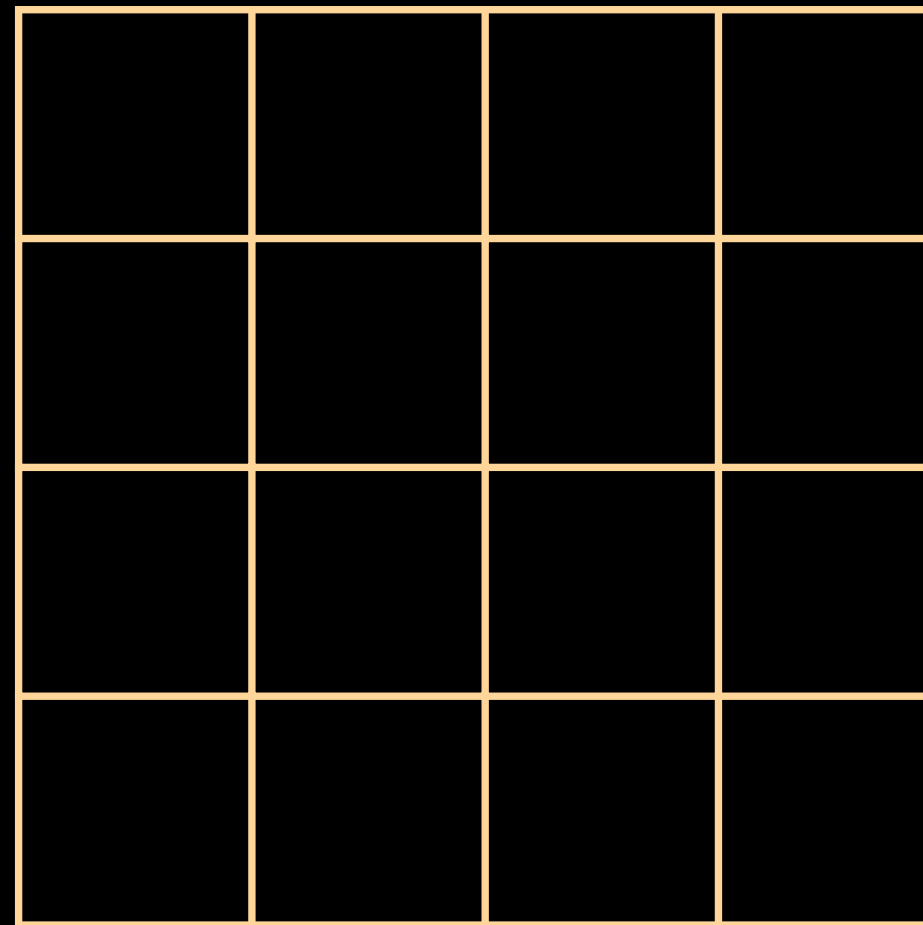
Ciphertext



Advanced Encryption Standard

Family of block ciphers standardized by NIST on 2001

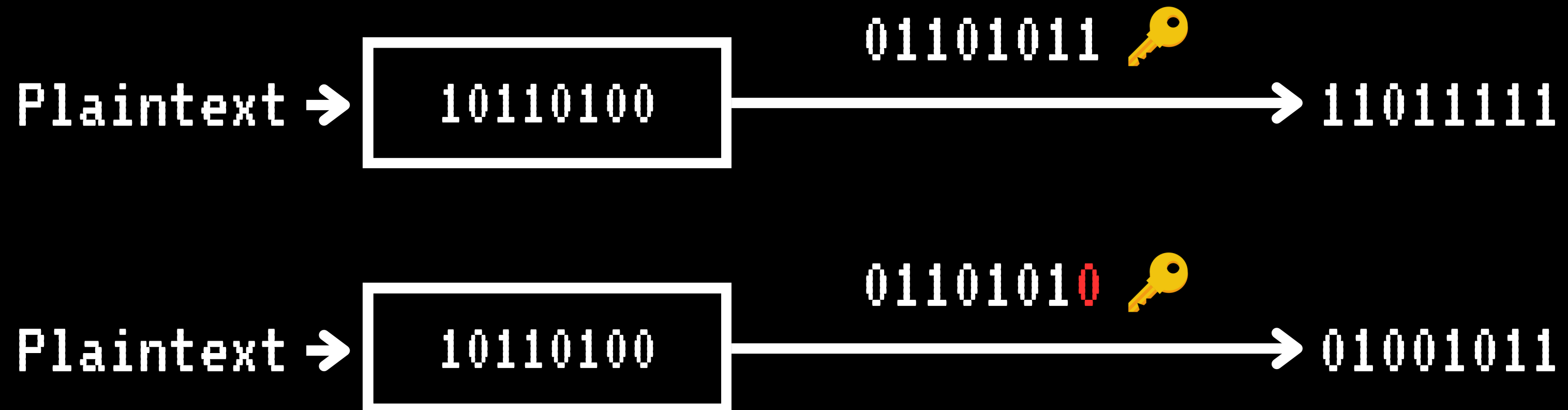
128 bits split into 16 bytes



Confusion

Properties of Symmetric-Key encryption, defined by Claude Shannon.

The relation between the Key and Ciphertext should be obscure.



Diffusion

Relation between the Plaintext and Ciphertext should be obscure.

Usually, if 1 bit in Plaintext changes, about half of the Ciphertext should change.



Data Encryption Standard

Published as an official FIPS standard in 1977

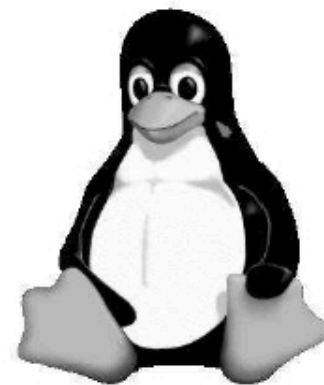
- No longer secure because it's key is only 56 bits.
- In modern hardware, DES can be brute forced in less than a day.
- In an effort to extend the life of DES, 3DES was proposed:

$$C = E_{DES}(k_1, D_{DES}(k_2, E_{DES}(k_3, p)))$$

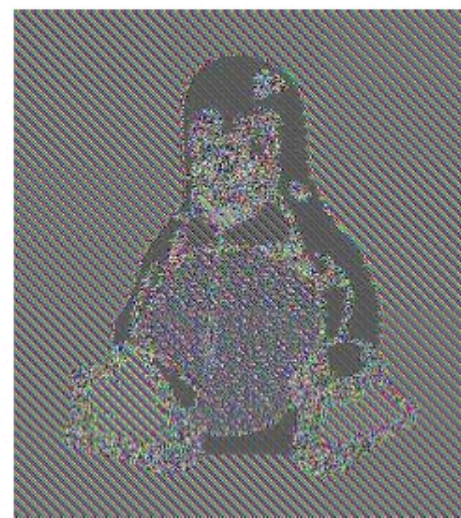
Block Cipher Modes of Operation

So far, Block ciphers can only send a messages of limited length. To send longer (ideally indefinite) length messages, we have to split the message into chunks.

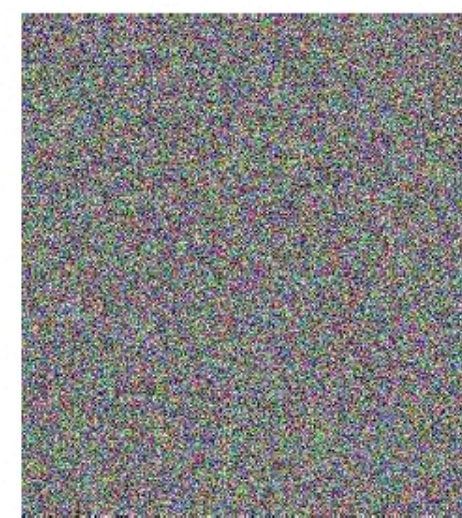
The problem with ECB



Tux (pixmap)

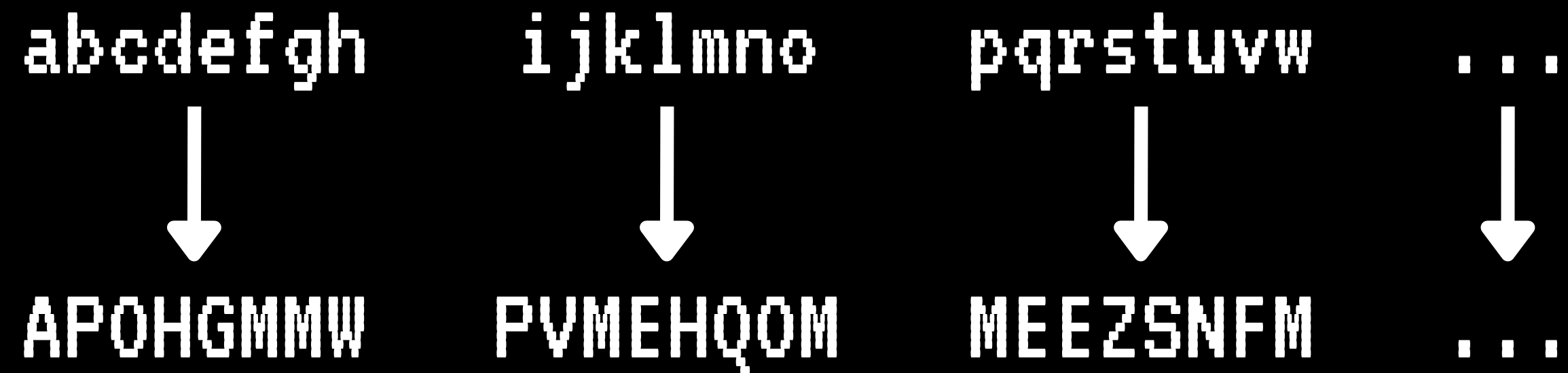


Encrypted with ECB



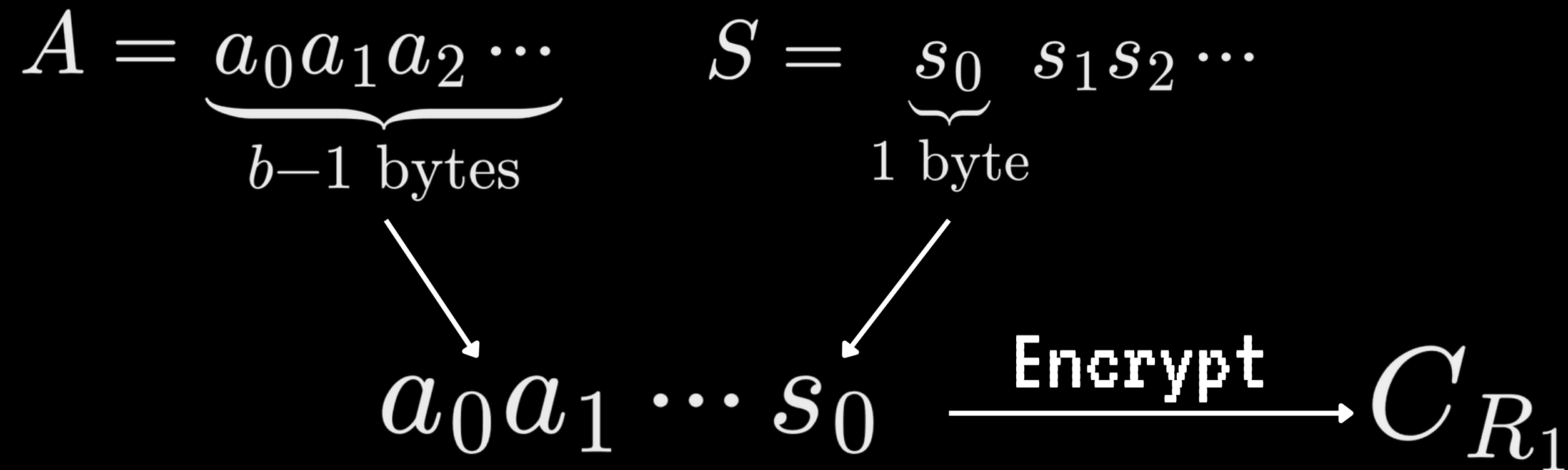
Encrypted with CFB

Electronic Codebook (ECB) Mode



ECB - Oracle Attack

Oracle: A system that performs encryption of a specific plaintext for the attacker

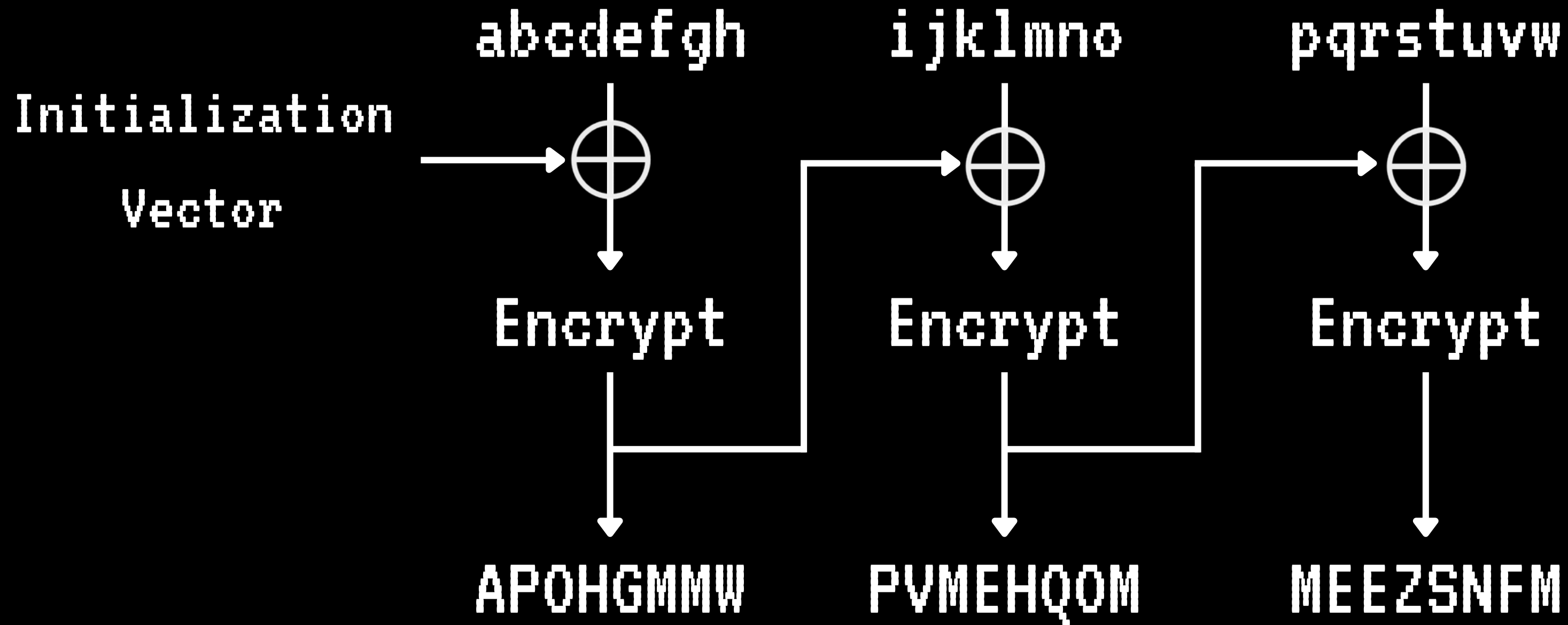


ECB - Oracle Attack

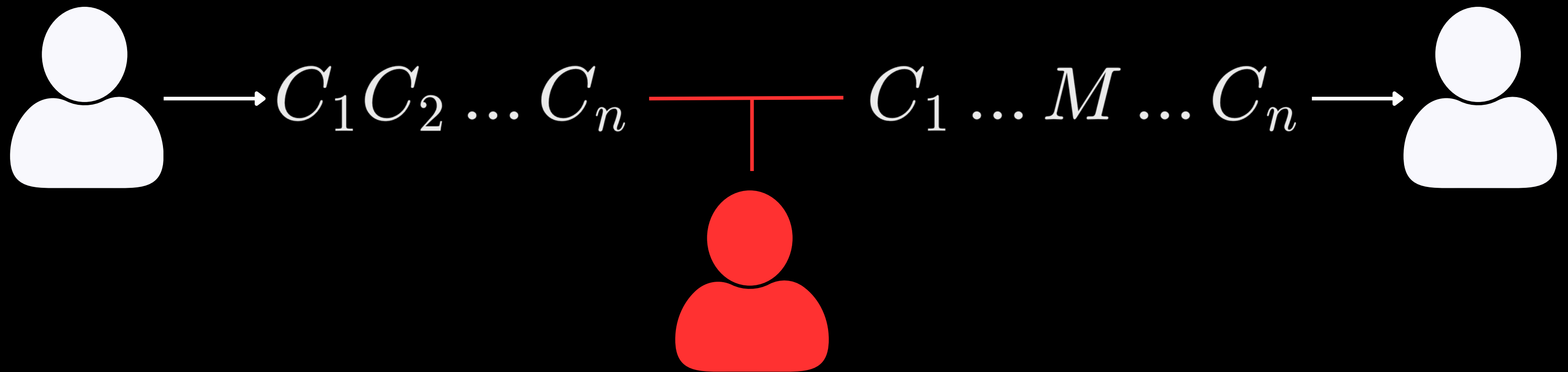
Typically, to brute force a block of 16 bytes,
there are 256^{16} combinations

This is reduced down to $256 \cdot 16 = 4096$
combinations with the Oracle Attack.

Cipher Block Chaining (CBC) Mode

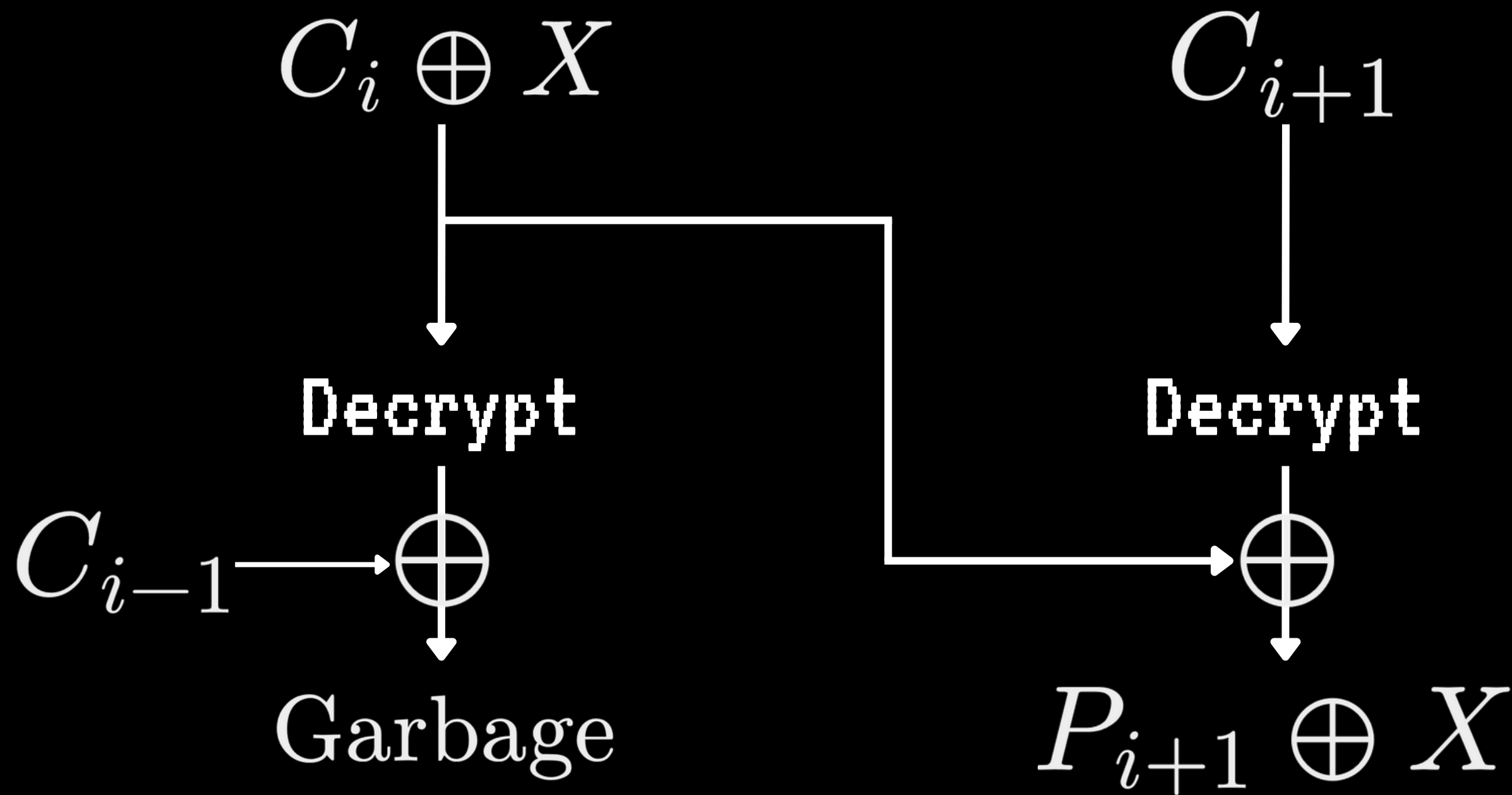


CBC - Bit Flipping Attack

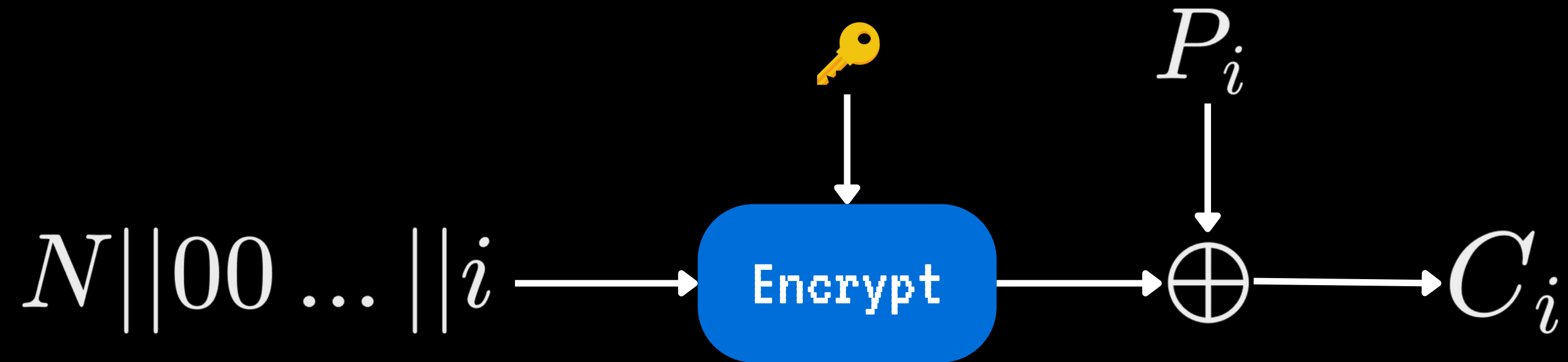


$$C_i \oplus X = M$$

CBC - Bit Flipping Attack



Counter (CTR) Mode



$N = \text{nonce}$

Asymmetric-Key Cryptography

Using one key to encrypt and one key to decrypt.

Public Key: Shared key for encrypting messages

Private Key: Private key for decrypting messages

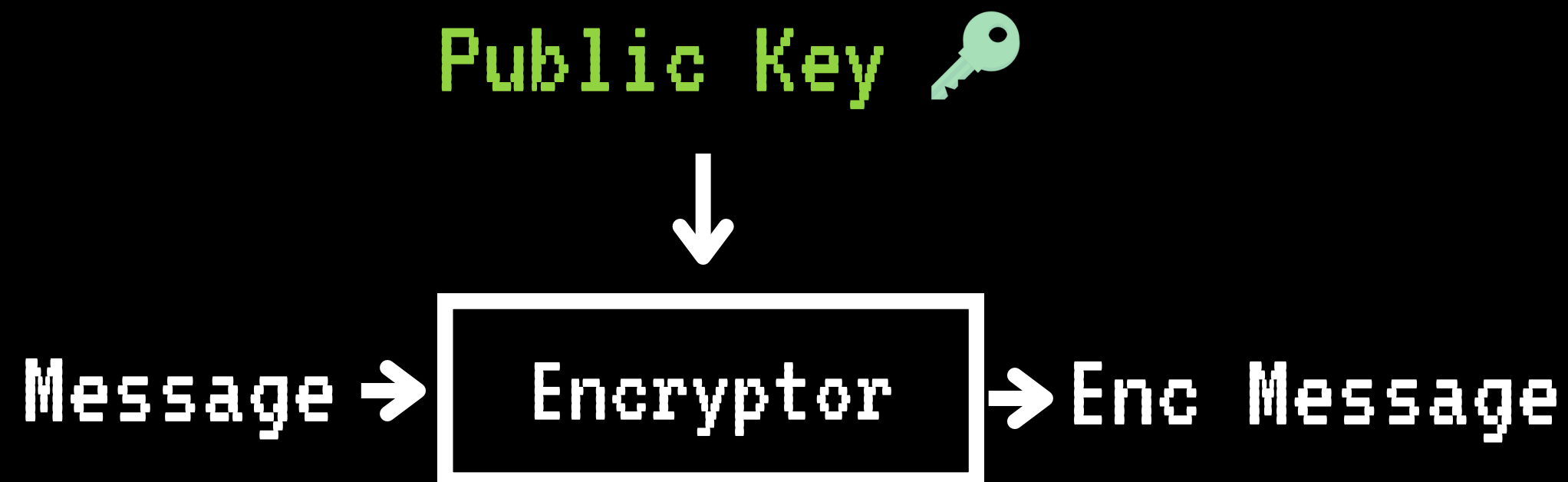
Public-Private Key Algorithms:

- RSA
- DHKE
- ECC

Asymmetric Crypto - Encrypting

Public Key: Shared key for encrypting messages

Private Key: Private key for decrypting messages



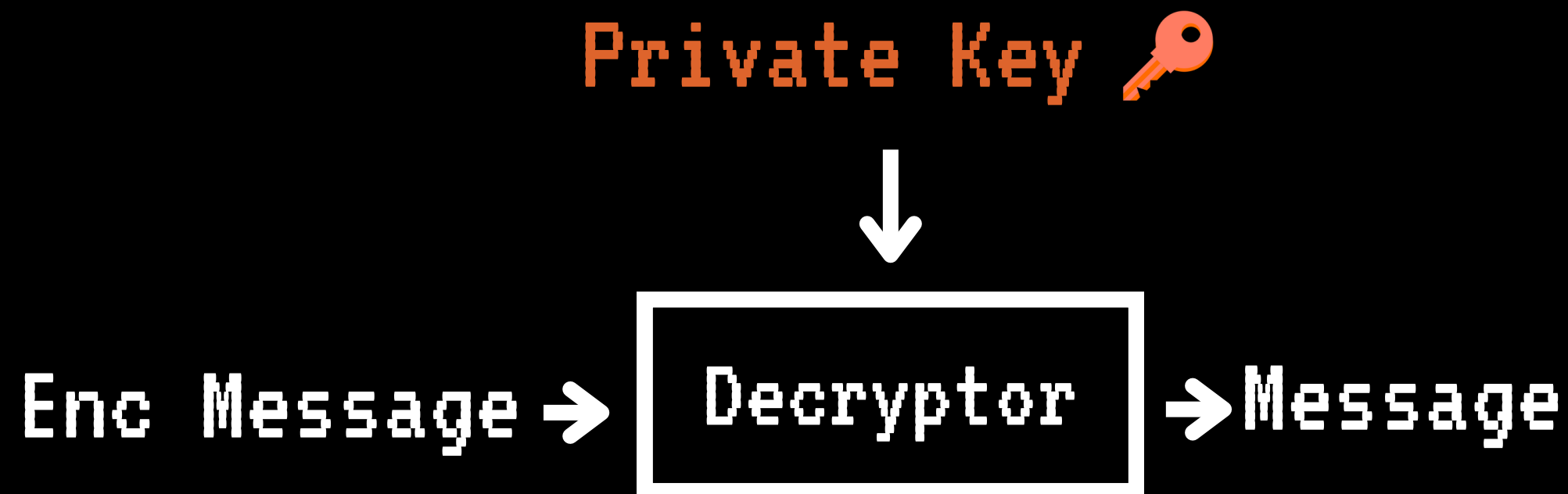
Alice sends Bob a message.

Alice encrypts her message with Bob's public key

Asymmetric Crypto - Decrypting

Public Key: Shared key for encrypting messages

Private Key: Private key for decrypting messages



Bob receives Alice's message.

Bob decrypts with his private key

Discrete Logarithm Problem (DLP)

$$a^x \equiv b \pmod{N}$$

Given: a, b, N

What is the value of x ?

Sidenote - Hard Problems

We assume specific problems in cryptography secure because there are no known solution (axioms of cryptography)

- Discrete Logarithm Problem
- ECC Discrete Logarithm Problem
- Short Vector Problem
- Learning With Errors Problem
- ...

* more specifically, there exists no probabilistic polynomial time algorithm to solve efficiently



RSA

An asymmetric cipher based off the hardness of DLP

Requires:

- p, q given prime numbers
- $n = p \times q$ product of 2 primes
- $\phi(n) = (p - 1) \times (q - 1)$ totient of 2 primes
- e public key, $\text{GCD}(e, \phi(n)) = 1$
- d private key, $e \times d \equiv 1 \pmod{\phi(n)}$

RSA

- p, q given prime numbers
- $n = p \times q$
- $\phi(n) = (p - 1) \times (q - 1)$
- e public key
- d private key

Encrypting:

$$\text{message}^e \bmod n = \text{cipher}$$

Decrypting:

$$\text{cipher}^d \bmod n = \text{message}$$

Using e, d to factor n

```
import random
import math

def factor_n_edn(e : int, d : int, n : int) -> tuple[int, int]:
    k = d * e - 1
    while True:
        g = random.randint(2, n - 1)
        t = k
        while t % 2 == 0:
            t = t // 2
        x = pow(g, t, n)
        if x == 1:
            break
        y = math.gcd(x - 1, n)
        if 1 < y < n:
            p = y
            q = n // y
            return (p, q)
```

proof: <https://crypto.stanford.edu/~dabo/papers/RSA-survey.pdf>

Using totient to factor n

```
sage: def crack_rsa(n, phi_n):  
.....:     R.<x> = PolynomialRing(QQ)  
.....:     f = x^2 - (n+1 - phi_n)*x + n  
.....:     return [b for b, _ in f.roots()]  
.....:  
sage: crack_rsa(31615577110997599711, 31615577098574867424)  
[8850588049, 3572144239]
```

Knowing totient, we can generate a
polynomial and p and q will be the roots

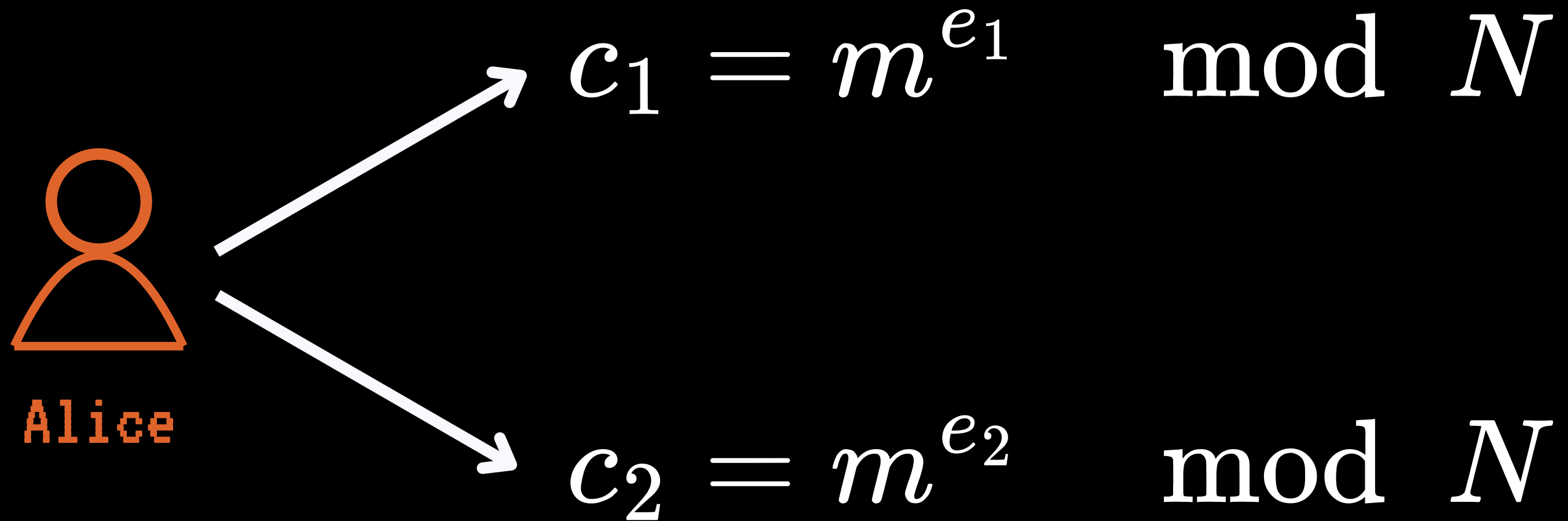
<https://www.youtube.com/watch?v=J5S00ncxybU>

* Sage is a superset of python to assist in math functions



Common Modulus Attack

If two messages use different keys but the same modulus N , they can break each other



Common Modulus Attack

$$c_1 = m^{e_1} \mod N$$

$$c_2 = m^{e_2} \mod N$$

* bézout's identity

$$\text{GCD}(e_1, e_2) \implies \exists a_1, a_2; a_1 e_1 + a_2 e_2 = 1$$

$$c_1^{a_1} \times c_2^{a_2} = (m^{e_1 \times a_1} \times m^{e_2 \times a_2}) \mod N$$

$$= (m^{e_1 \times a_1 + e_2 \times a_2}) \mod N$$

$$= (m^1) \mod N$$

$$= m$$

Common Modulus Attack

```
import math

def egcd(a, b):
    if a == 0:
        return (b, 0, 1)
    else:
        g, y, x = egcd(b % a, a)
        return (g, x - (b // a) * y, y)

def modinv(a, m):
    g, x, y = egcd(a, m)
    if g != 1:
        raise ValueError('Modular inverse does not exist.')
    else:
        return x % m

def attack(c1, c2, e1, e2, N):
    if math.gcd(e1, e2) != 1:
        raise ValueError("Exponents e1 and e2 must be coprime")
    s1 = modinv(e1, e2)
    s2 = (math.gcd(e1, e2) - e1 * s1) / e2
    temp = modinv(c2, N)
    m1 = pow(c1, s1, N)
    m2 = pow(temp, -s2, N)
    return (m1 * m2) % N
```

<https://scribe.rip/rsa-attacks-common-modulus-7bdb34f331a5>

Wiener's Attack

Wiener attacks are effective if your d is small.

Uses continued fractions

```
import math

def continued_fraction_expansion(numerator, denominator):
    cf = []
    while denominator:
        a = numerator // denominator
        cf.append(a)
        numerator, denominator = denominator, numerator - a * denominator
    return cf

def convergents_from_cf(cf):
    convergents = []
    for i in range(len(cf)):
        if i == 0:
            convergents.append((cf[0], 1))
        elif i == 1:
            h = cf[0] * cf[1] + 1
            k = cf[1]
            convergents.append((h, k))
        else:
            h1, k1 = convergents[i - 1]
            h2, k2 = convergents[i - 2]
            h = cf[i] * h1 + h2
            k = cf[i] * k1 + k2
            convergents.append((h, k))
    return convergents
```

```
def is_perfect_square(n):
    if n < 0:
        return None
    root = int(math.isqrt(n))
    return root if root * root == n else None

def wiener_attack(e, n):
    cf = continued_fraction_expansion(e, n)
    convs = convergents_from_cf(cf)
    for k, d_candidate in convs:
        if k == 0:
            continue
        if (e * d_candidate - 1) % k != 0:
            continue
        phi_candidate = (e * d_candidate - 1) // k
        a = 1
        b = -(n - phi_candidate + 1)
        c = n
        discriminant = b * b - 4 * a * c
        sqrt_discriminant = is_perfect_square(discriminant)
        if sqrt_discriminant is None:
            continue
        x1 = (-b + sqrt_discriminant) // 2
        x2 = (-b - sqrt_discriminant) // 2
        if x1 * x2 == n:
            return d_candidate # Found the private key!
    return None # Attack failed
```

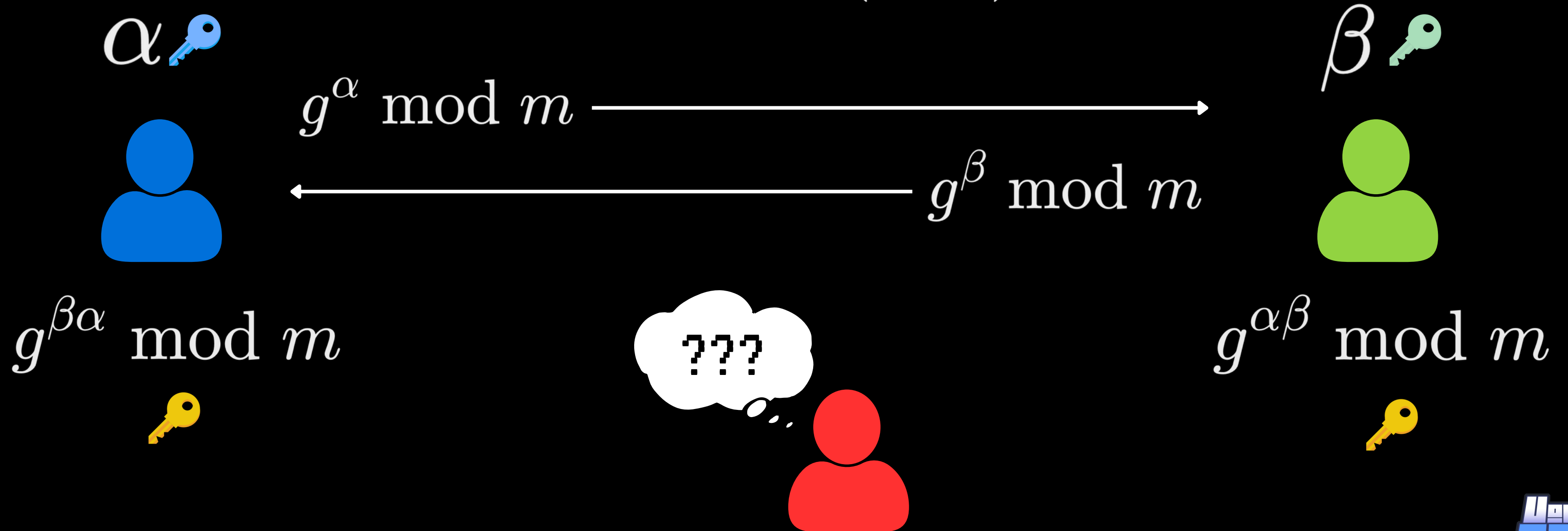
<https://scribe.rip/@it7th/how-wieners-attack-breaks-rsa-when-you-pick-a-weak-private-key-7113fda6d969>



Diffie-Hellman Key Exchange

How do we share the **secret** key?

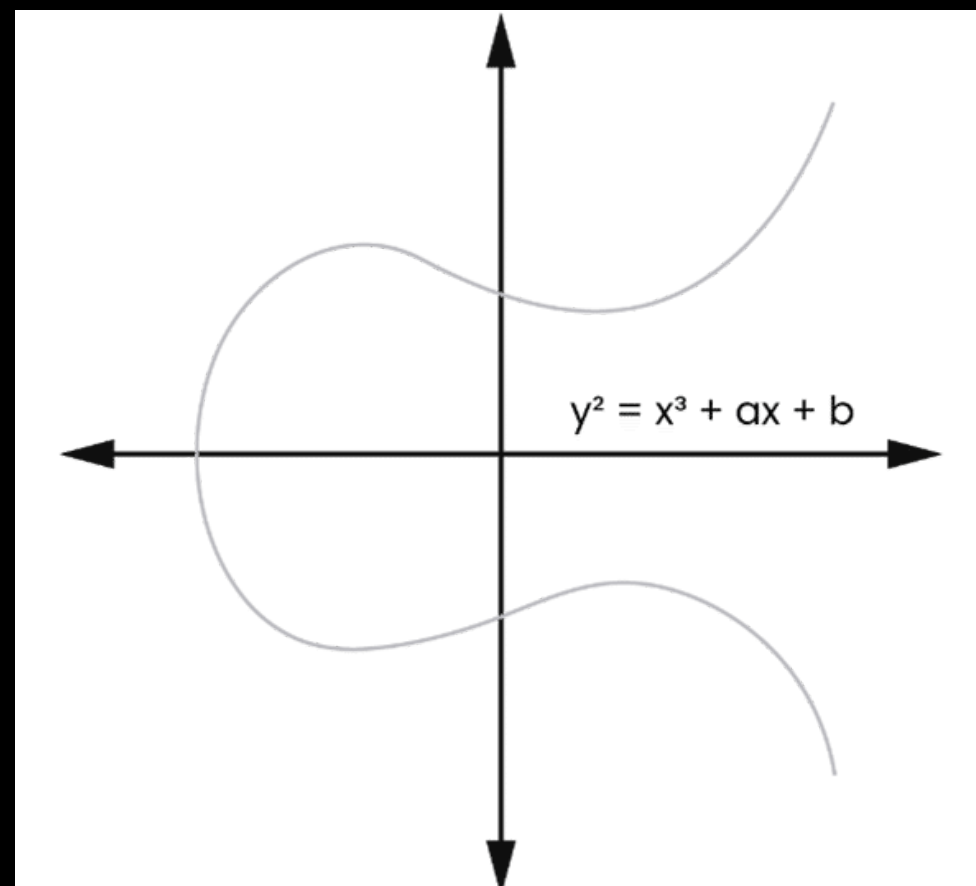
Public Key: (g, m)



Elliptic Curve Cryptography

Public-Key cryptography system that relies on elliptic curve discrete logarithm problem (ECDLP).

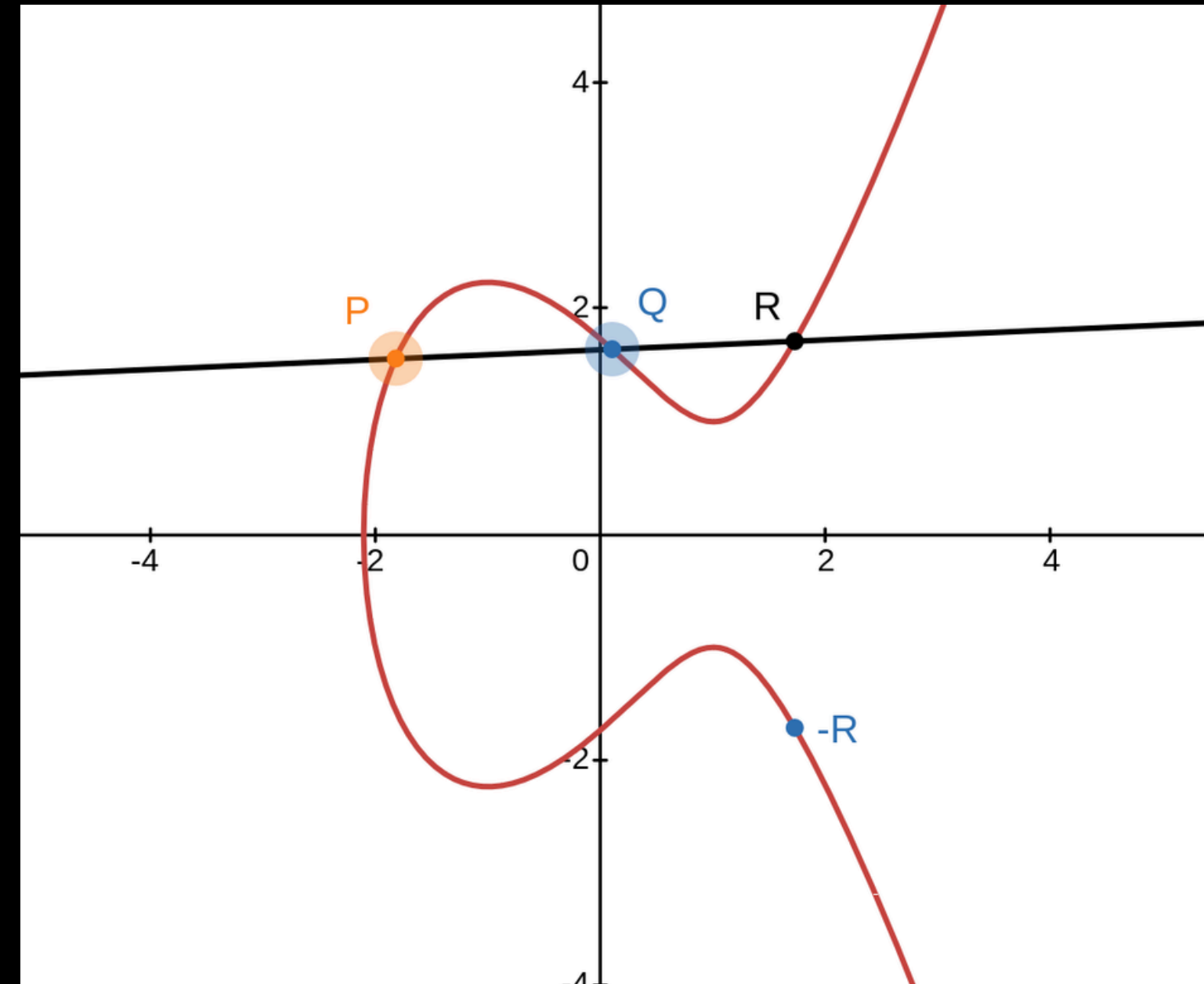
Used in iMessage, Bitcoin, Ethereum, etc.



Elliptic Curves

Elliptic Curves has
specific algebraic laws
which form a Group.

$$P + Q = -R$$



ECDLP

Given P, Q such that

$$Q = kP = \underbrace{P + P + \dots P}_{k \text{ times}}$$

Can we find k ?

Table 1. Intuitive security levels.

security level	volume of water to bring to a boil	bit-lengths		
		symmetric key	cryptographic hash	RSA modulus
teaspoon security	0.0025 liter	35	70	242
shower security	80 liter	50	100	453
pool security	2 500 000 liter	65	130	745
rain security	0.082 km ³	80	160	1130
lake security	89 km ³	90	180	1440
sea security	3 750 000 km ³	105	210	1990
global security	1 400 000 000 km ³	114	228	2380
solar security	-	140	280	3730

Hashing

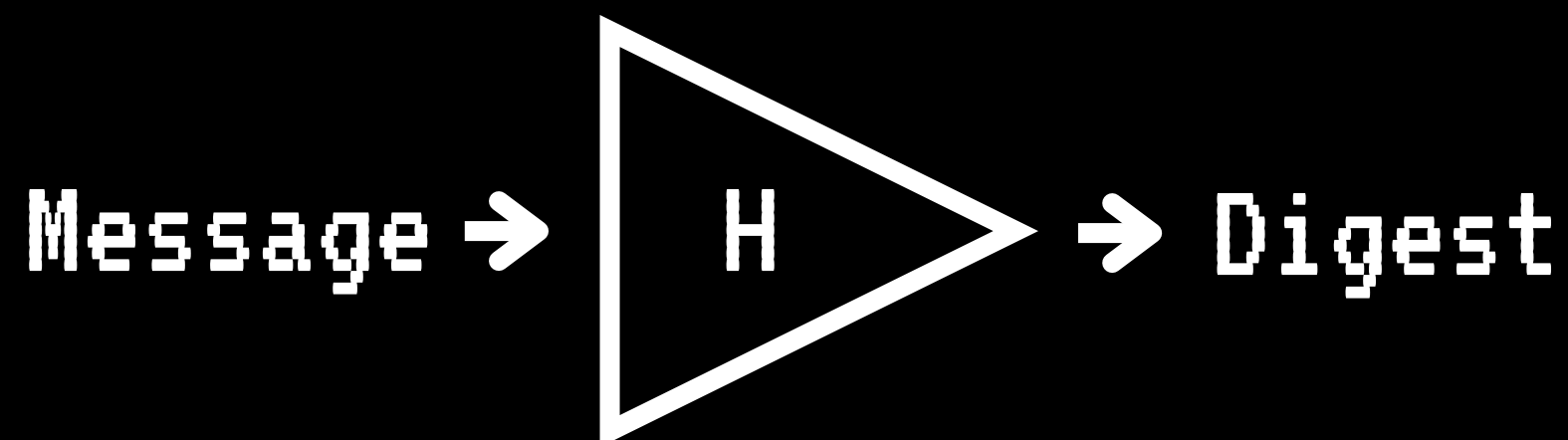
Hashing

A cryptographic function that takes an input message and returns a output message (Hash/Digest) that is:

- Non-reversible
- Fixed-size
- Unique (no collisions)

Example algorithms:

- SHA256
- MD5



Hashing

Hashes are used for integrity checks.

* used for Randomness aswell, can also be used for a variety of non-security things (hash table, compression, etc)

mouse72 → SHA1 → 0752511799b89e35fcbf975fdb3f820354b7409b

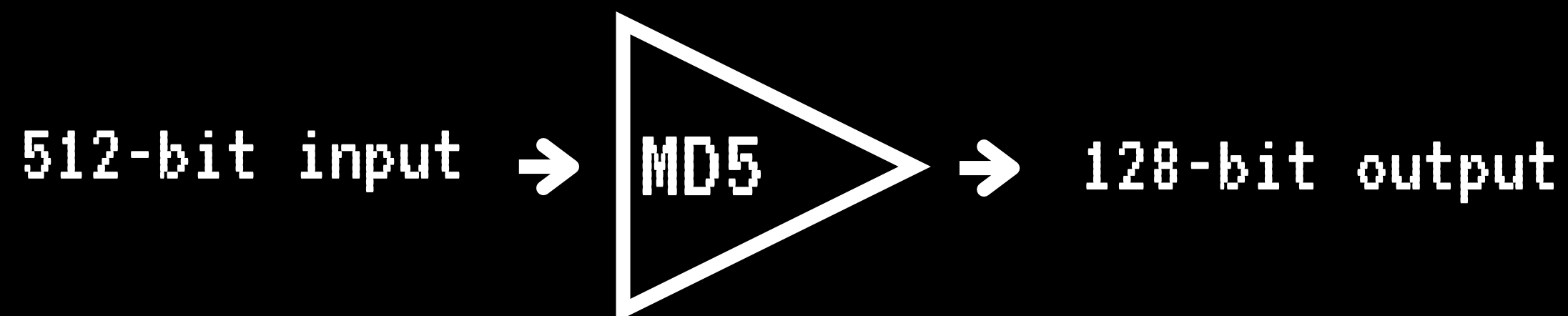
mouse73 → SHA1 → d227f2c750b1ddf474999c914aee334ba055ab49

Different messages produce completely different hashes



Message Digest 5 (MD5)

Very quick hashing algorithm. Not secure



If input is smaller, it is padded

If input is larger, we run MD5 several times for each 512-bit chunk

MD5 High Level Algorithm

- MD5 starts with hardcoded arrays $K[]$, $r[]$ and a hardcoded state:
 - $A = 0x67452301$
 - $B = 0xefcdab89$
 - $C = 0x98badcfe$
 - $D = 0x10325476$
- MD5 takes 512-bit input and splits into 16 chunks to make a array
- MD5 performs 4 rounds:
 - Round 1: iterate through with $i = (i+1)$, performing $\text{round_func}(i)$
 - Round 1: iterate through with $i = (5*i+6)\%16$, performing $\text{round_func}(i)$
 - Round 1: iterate through with $i = (3*i+5)\%16$, performing $\text{round_func}(i)$
 - Round 1: iterate through with $i = (7*i)\%16$, performing $\text{round_func}(i)$
- After all rounds, $A||B||C||D$ is the output ($||$ is string concat)

MD5 - K, r

Array K (xor array)

64
elements

0xd76aa478
0xe8c7b756
0x242070db
0xc1bdcee5
...
0xeb86d391

Array r (left rotate array)

64
elements

7
12
17
22
...
21

These arrays are generated by functions, but are always constant

MD5 - Round Function

Plaintext Array (we split plaintext into 16 chunks)

$\boxplus$: addition in mod 2^{32}

0xafab2cb9	0x01ef4202	0xad105b70	0xba234427	0xb4de9f19	0xafab2cb9	0xa2690ac4	0x49d4189c	0x608c7f60	0x956b12af	0x542f0088	0x47e57aae	0x2ab2d30a	0xf0665918	0x25524ce1	0x07da7f62
------------	------------	------------	------------	------------	------------	------------	------------	------------	------------	------------	------------	------------	------------	------------	------------

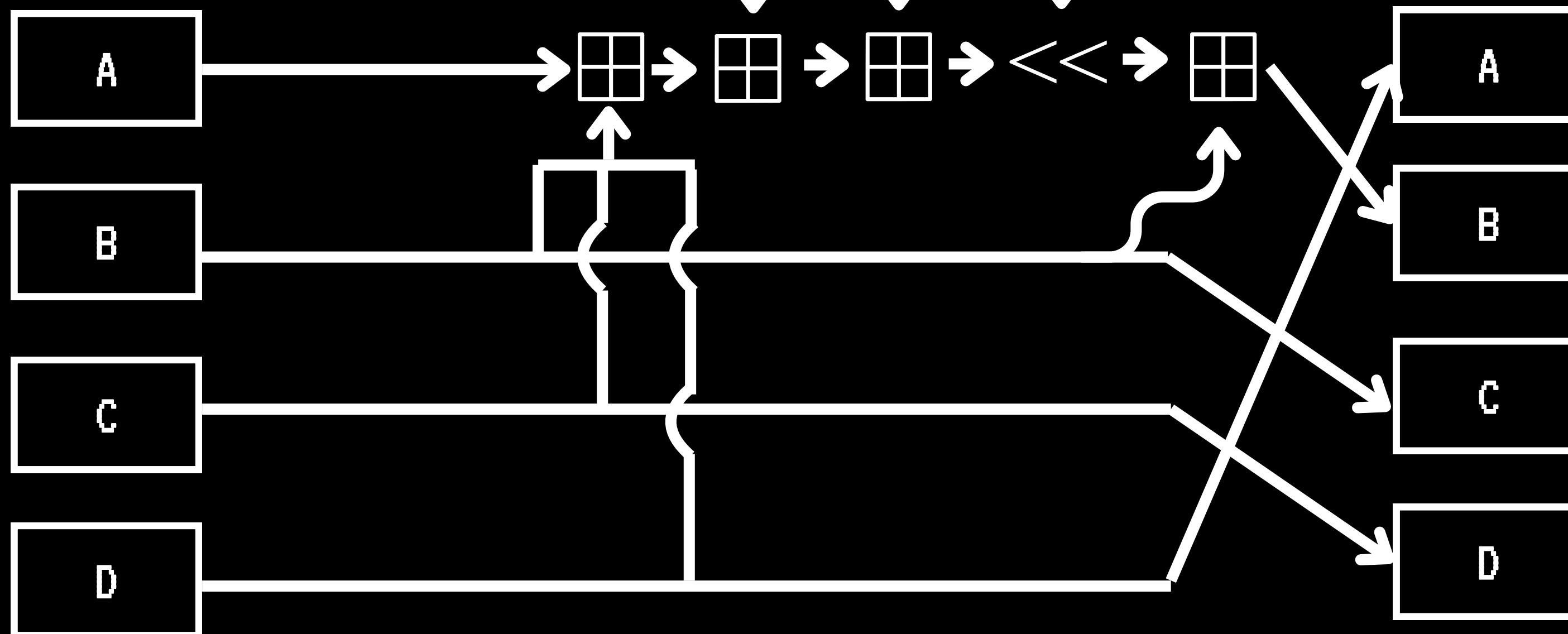
Current State

Plaintext[i]

K[i]

r[i]

Next State



Hash Collision

When different inputs produce the same hash digest

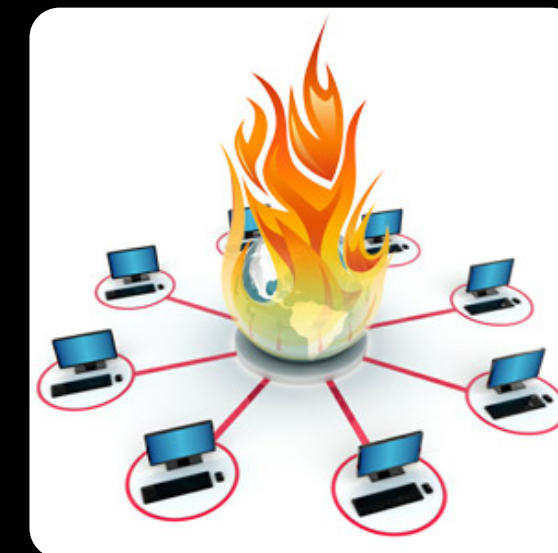


If you rely on hashes for integrity, then a hash collision is dangerous

MD5 Hash Collision

MD5 has a hash collision resistance of 2^{18} operations (not very good)

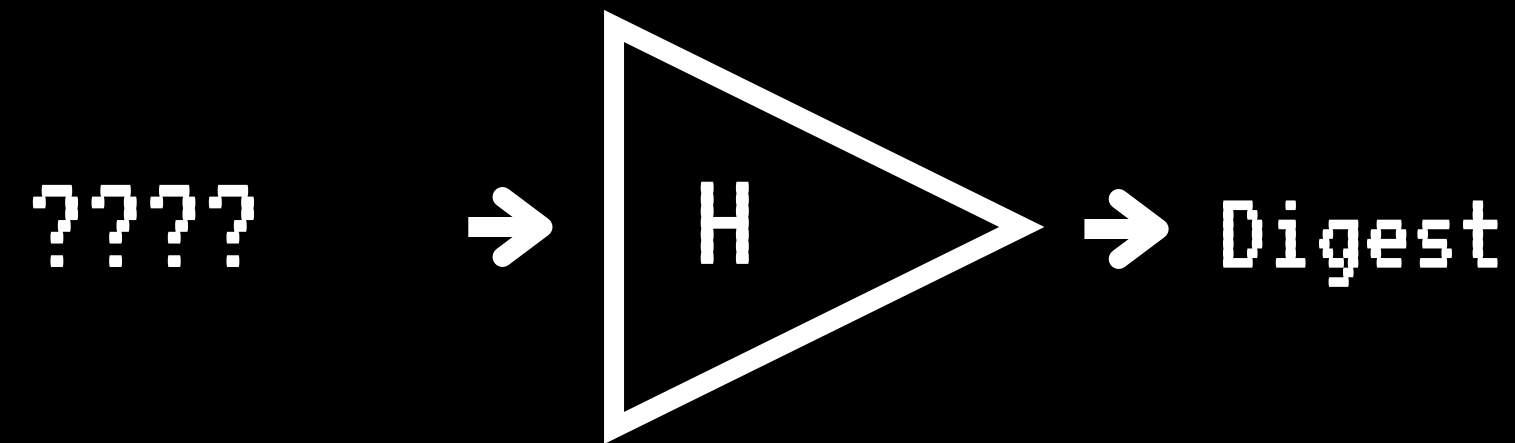
In 2012, Flame malware used MD5 collision to
forge windows code-signing certificate



Preimage Attack

Techniques to find inputs corresponding to specific hash outputs.

Often done via refined brute force.

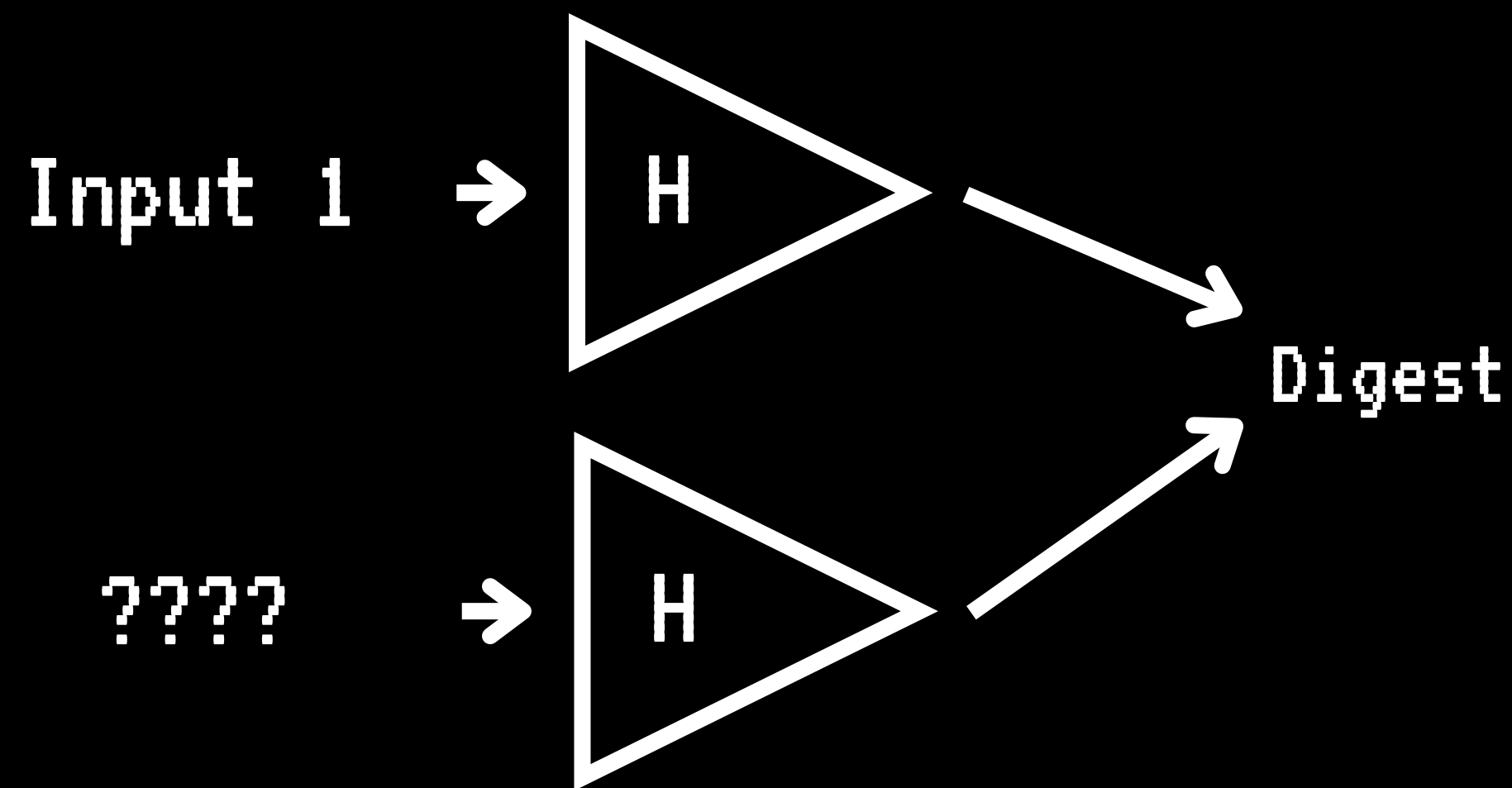


Preimage attacks are very hard to perform practically
unless you have a supercomputer.

A good hash function should be preimage resistant

Second Preimage Attack

Technique where attacker is given an input and its hash, and tries to find another input with the same hash.



A good hash function should be second preimage resistant

* Git, which uses SHA-1 as its integrity hash does not care about second pre-image resistance, as it checks dates in the case of same hashes

Secure Hashing Algorithm

A family of hash functions most common for secure hashing today

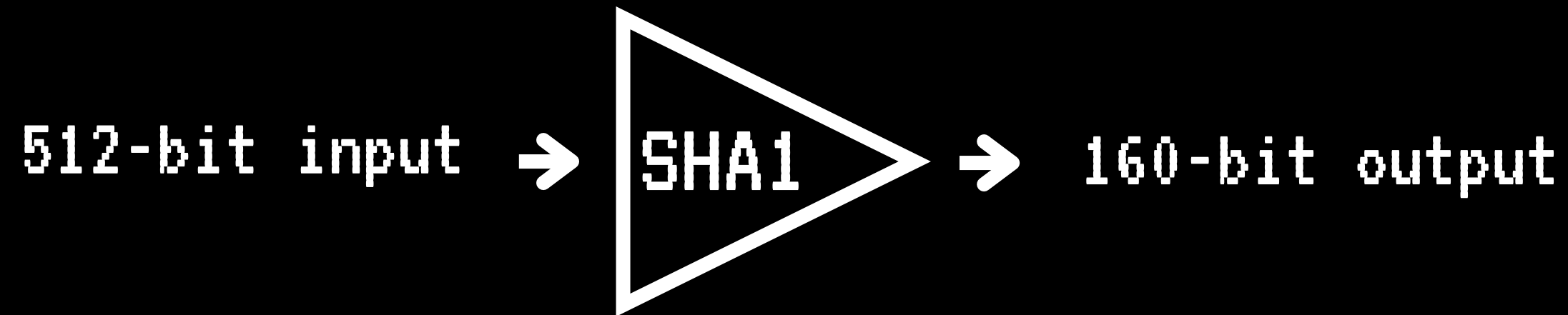
- SHA-0 (not used)
- SHA-1
- SHA-2 (SHA-256/SHA-512)
- SHA-3



no backdoors hopefully

SHA1

Revision of SHA-0. Not theoretically secure - supercomputers will break it



If input is smaller, it is padded

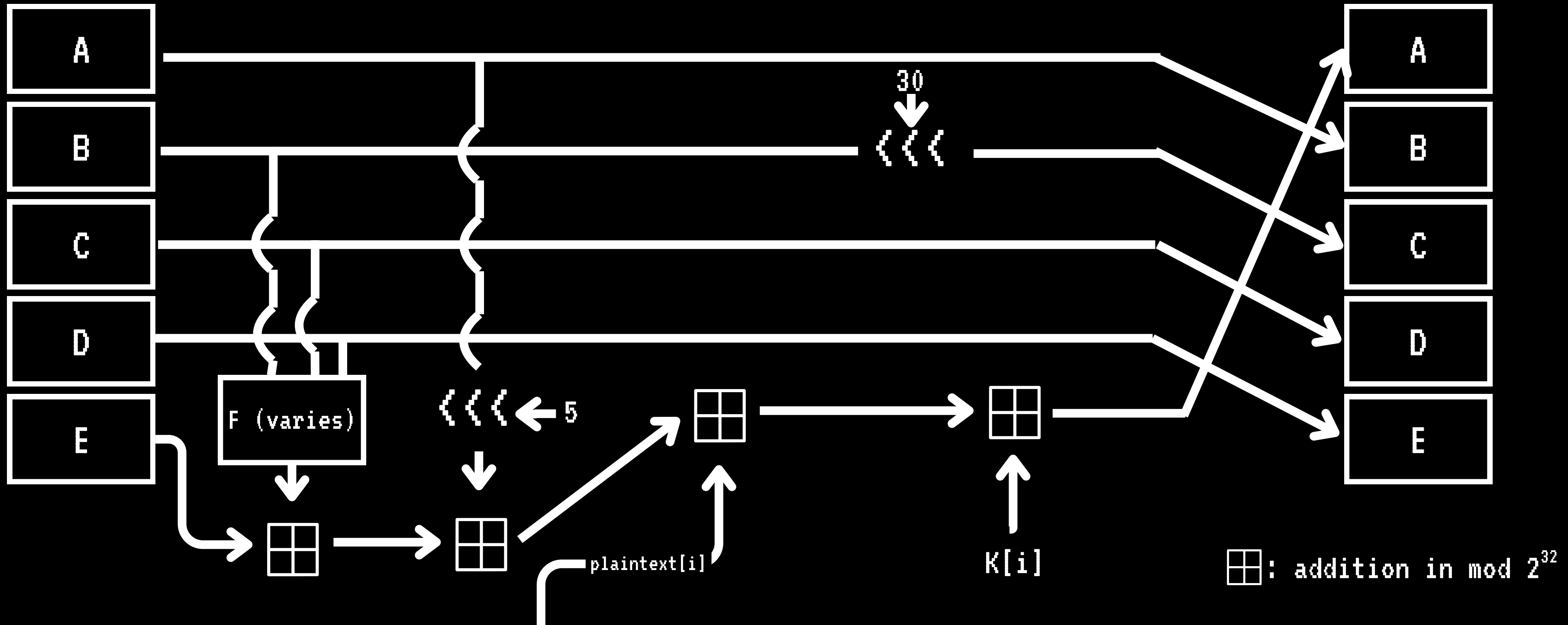
If input is larger, we run SHA1 several times for each 512-bit chunk

Slightly more expensive than MD5

SHA1 - Algorithm

Current State

Next State



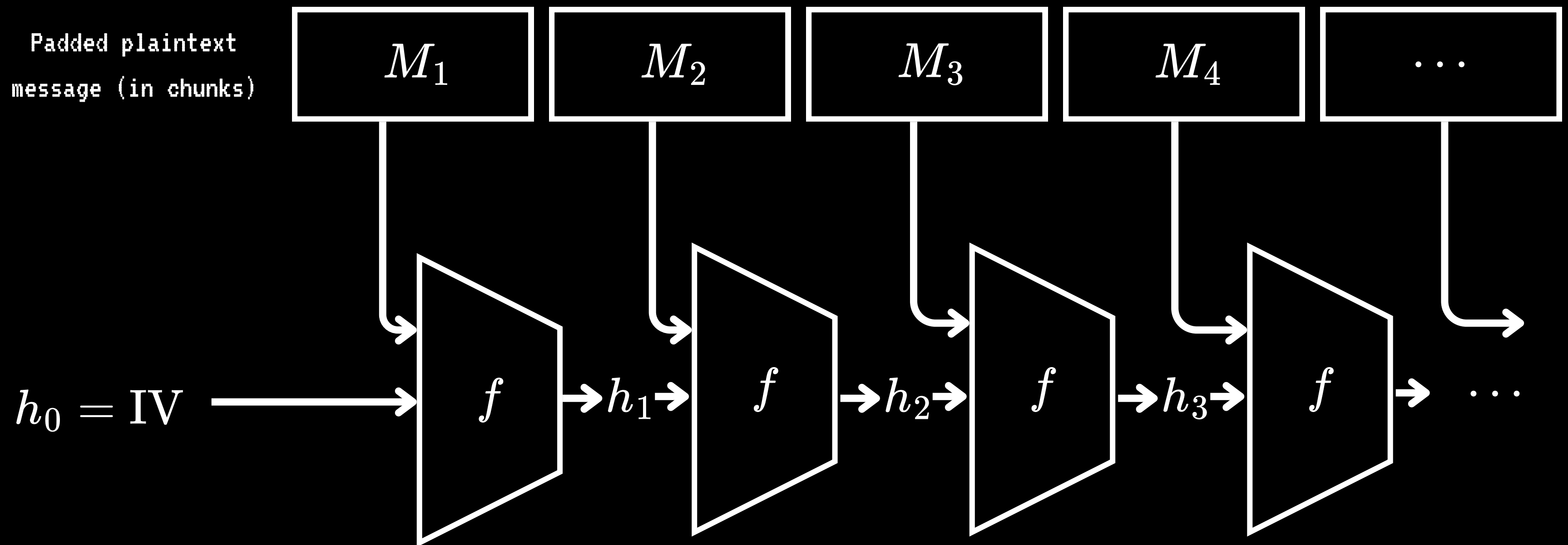
0xafab2cb9	0x01ef4202	0xad105b70	0xba234427	0xb4de9f19	0xafab2cb9	0xa2690ac4	0x49d4189c	0x608c7f60	0x956b12af	0x542f0088	0x47e57aae	0x2ab2d30a	0xf0665918	0x25524ce1	0x07da7f62
------------	------------	------------	------------	------------	------------	------------	------------	------------	------------	------------	------------	------------	------------	------------	------------

Plaintext Array (we split plaintext into 16 chunks)

Sidenote - Merkle Damgard Construction

A method of designing collision-resistant hash functions

Compression function f : Takes two inputs, return fixed size output



MD5, SHA-1, SHA-2 are all based off this

Length Extension Attack

Vulnerability in Merkle Damgard Hashes

Given:

- The hash of the message $\text{Hash}(\text{message1})$
- The length of the message $\text{Length}(\text{message1})$

Then:

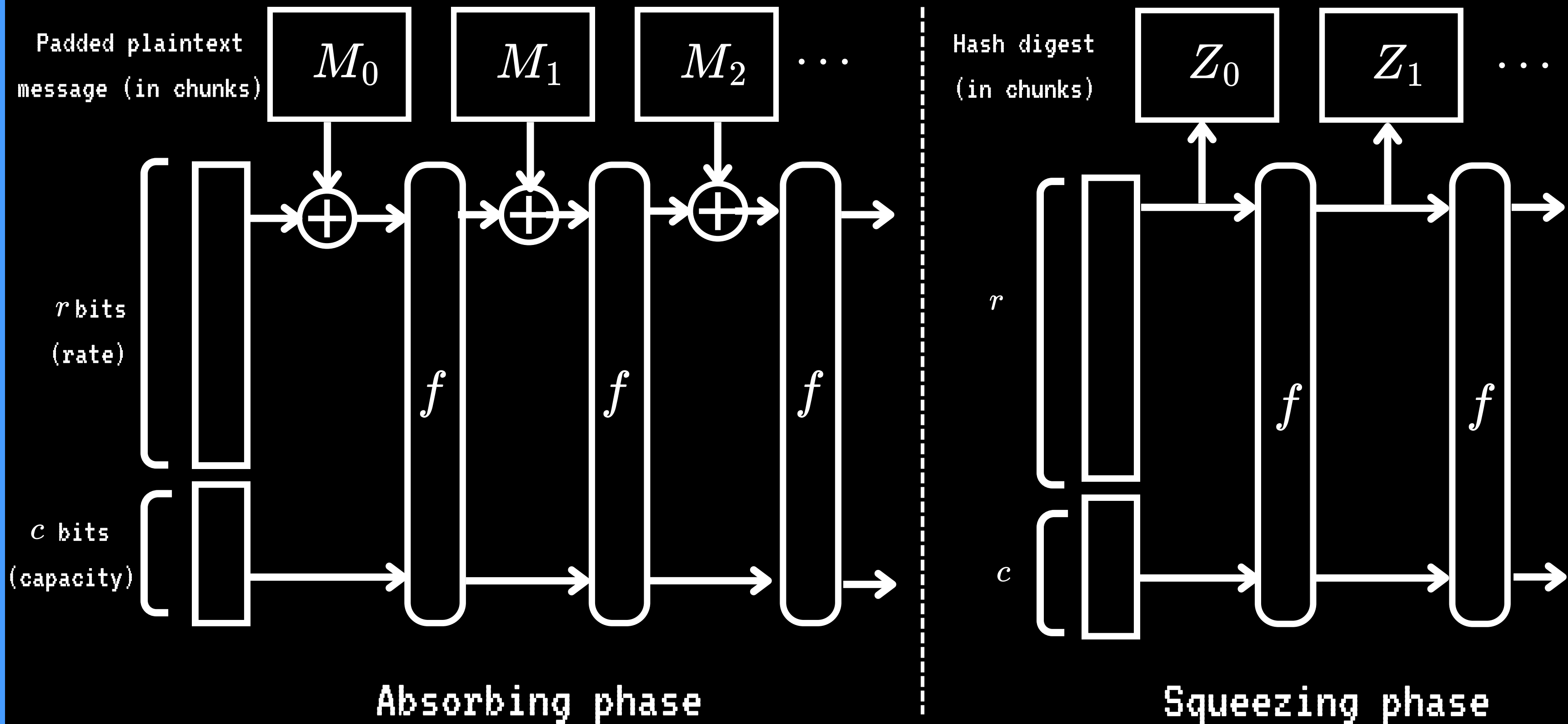
- An attacker can generate: $\text{Hash}(\text{message1}||\text{message2})$
(the hash of the message concatenated with arbitrary bytes)

* This can be problematic when we talk about MACs later



Sidenote - Sponge Function Construction

An alternative to Merkle Damgard construction. Not vuln to length extension

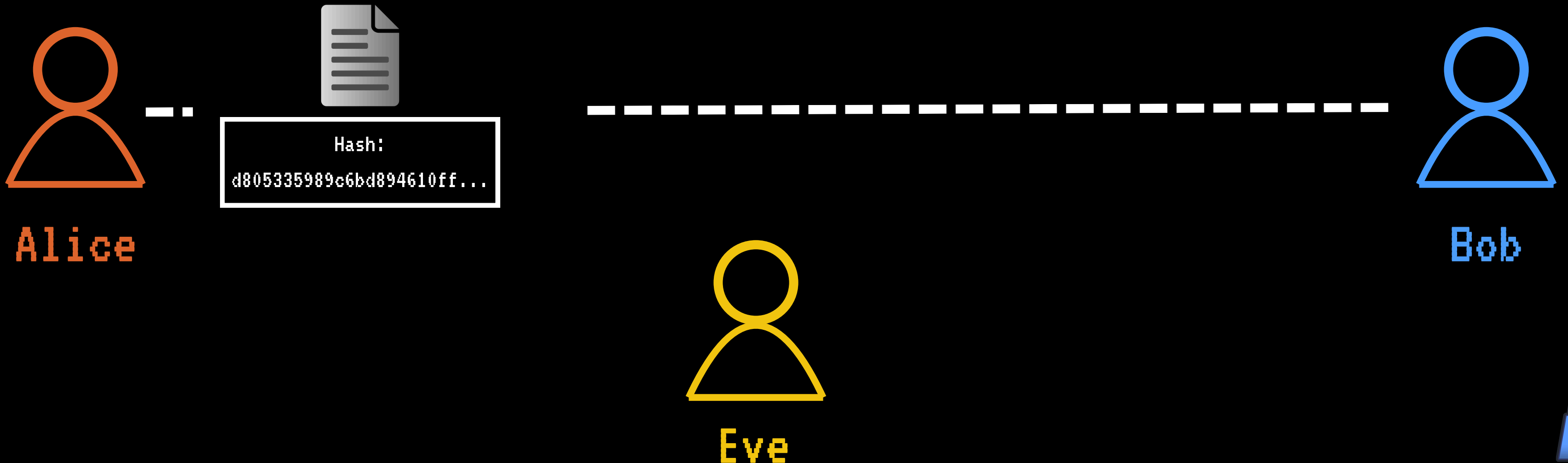


Message Authentication Codes

Hashes are not all! MITM Attack

If we want to send messages across a network and ensure they are not being tampered with, checking hashes alone does not work!

Alice will send message + hash to Bob



Hashes are not all! MITM Attack

If we want to send messages across a network and ensure they are not being tampered with, checking hashes alone does not work!

Eve intercepts and changes the msg + hash

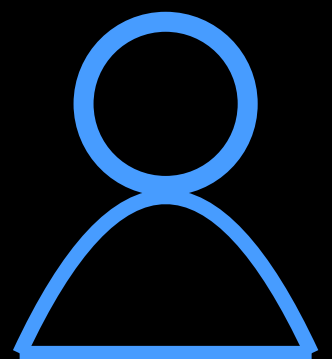


Alice



Hash:

217f111f6044be168e8882...



Bob



Eve

Hashes are not all! MITM Attack

If we want to send messages across a network and ensure they are not being tampered with, checking hashes alone does not work!

Bob verifies and the evil hash matches the evil msg

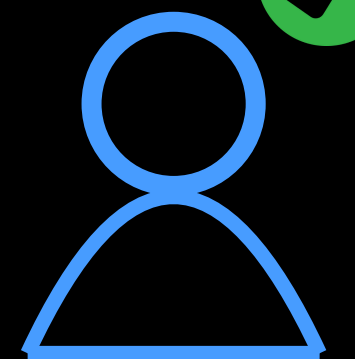


Alice



Hash:

217f111f6044be168e8882...



Bob

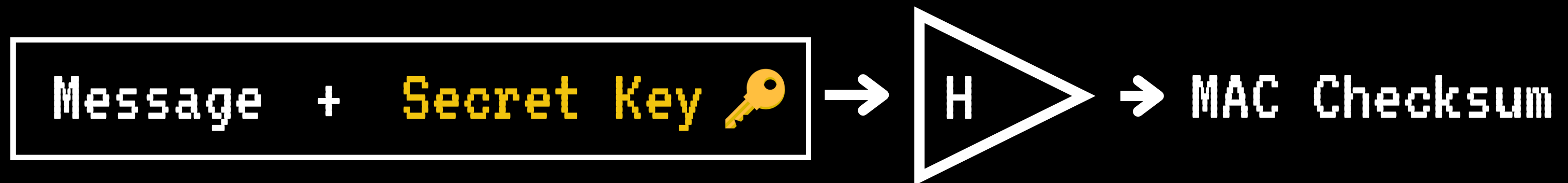


Eve

Message Authentication Codes

Hashing a message and its secret key to allow for post-transit:

- Integrity check
- Authentication



MAC Communication

If we use a MAC, we can verify integrity after transit

Alice will send combine message + key to
make a MAC, then send msg + MAC to Bob



MAC Communication

If we use a MAC, we can verify integrity after transit

Eve intercepts, changes msg and MAC



MAC Communication

If we use a MAC, we can verify integrity after transit

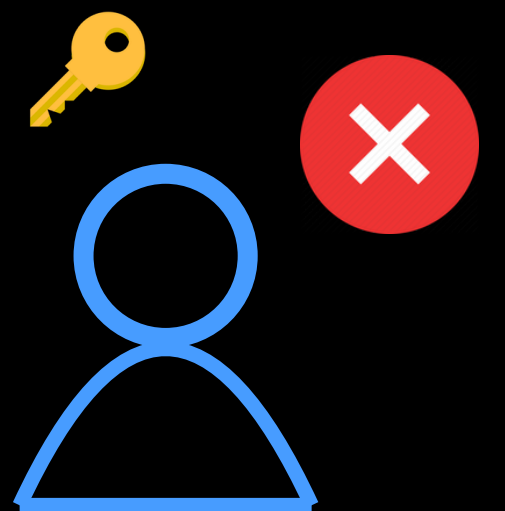
Bob compares msg + key, and finds the MAC
is different. He does not accept this msg



Alice



MAC:
427f111f6044be168e8882...



Bob



Eve

MAC from Hash

How do we combine a message and a key? Hash function by itself is too
ambiguous

$$\text{Hash}(\text{message}||\text{key}) \neq \text{Hash}(\text{key}||\text{message})$$

🔑 **paswort12** + tardigrade → SHA-256 → 1f9b01511d47cf9509b935b4
(Secret prefix) acf07488a77a5af2b3de59a9
a95939286508c88a

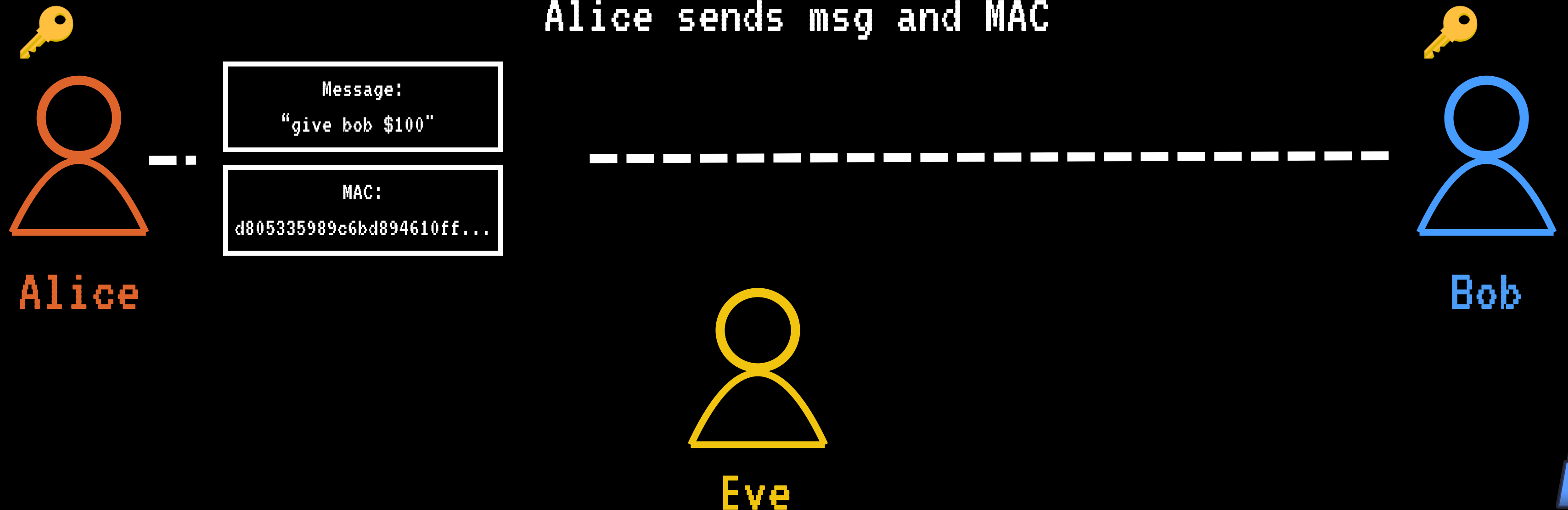
tardigrade + 🔑 **paswort12** → SHA-256 → 2c7b76466130932dda297320
(Secret suffix) c0a90e34e1e9fc7f763c9a95
8fd343bc1e21a75b

Secret Prefix Length Extension Attack

Lets say you choose to implement MAC via secret prefix method:

$$\text{MAC}_{\text{key}}(\text{message}) = \text{Hash}(\text{key} || \text{message})$$

If you use SHA-1, SHA-2, or any Merkle Damgard Construction hash...



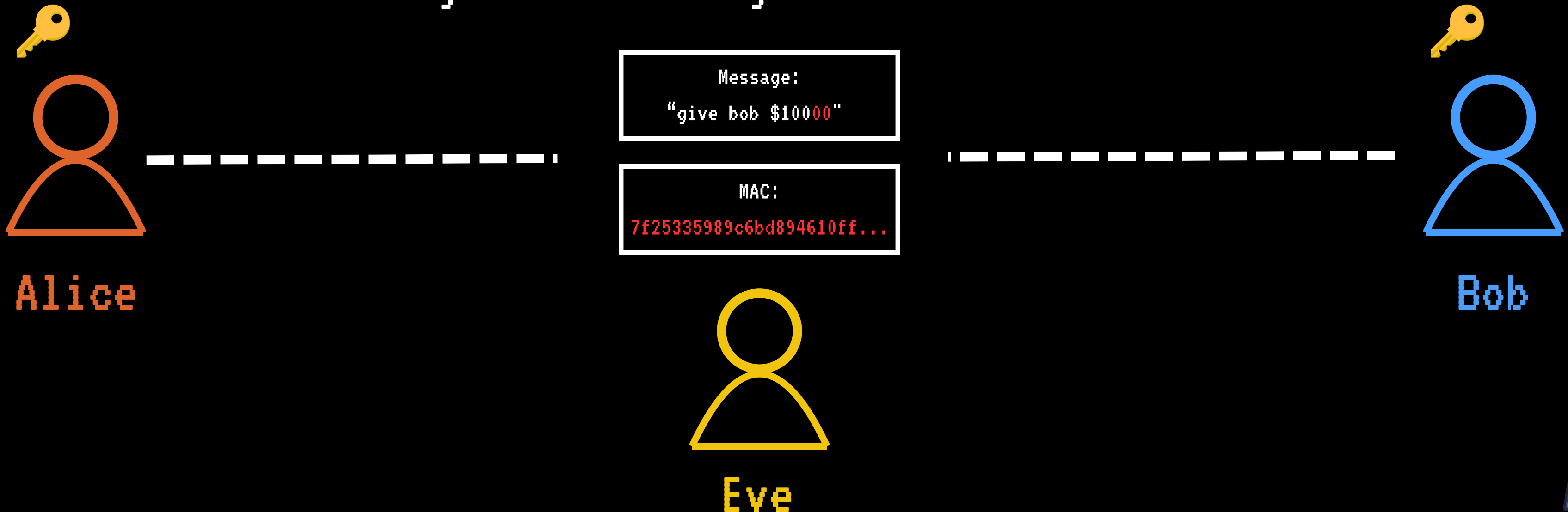
Secret Prefix Length Extension Attack

Lets say you choose to implement MAC via secret prefix method:

$$\text{MAC}_{\text{key}}(\text{message}) = \text{Hash}(\text{key}||\text{message})$$

If you use SHA-1, SHA-2, or any Merkle Damgard Construction hash...

Eve extends msg AND uses length-ext attack to overwrite hash



Secret Prefix Length Extension Attack

Lets say you choose to implement MAC via secret prefix method:

$$\text{MAC}_{\text{key}}(\text{message}) = \text{Hash}(\text{key} || \text{message})$$

If you use SHA-1, SHA-2, or any Merkle Damgard Construction hash...

Bob verifies msg+key hash is same as MAC

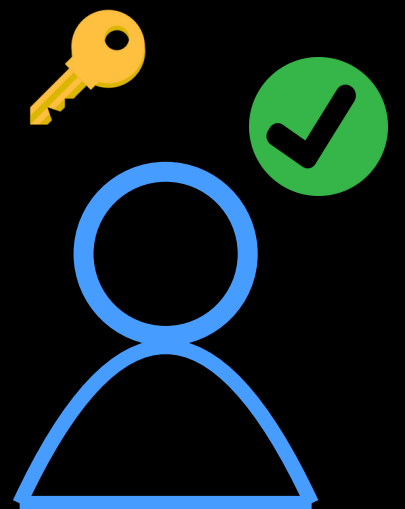


Alice



Message:
"give bob \$10000"

MAC:
7f25335989c6bd894610ff...



Bob



Eve

Because the message is at
the end, length ext attacks
work

Secret Suffix Forgery Attack

Lets say you choose to implement MAC via secret suffix method:

$$\text{MAC}_{\text{key}}(\text{message}) = \text{Hash}(\text{message}||\text{key})$$

If an attacker can find a hash collision for the message, they can keep the
same MAC.

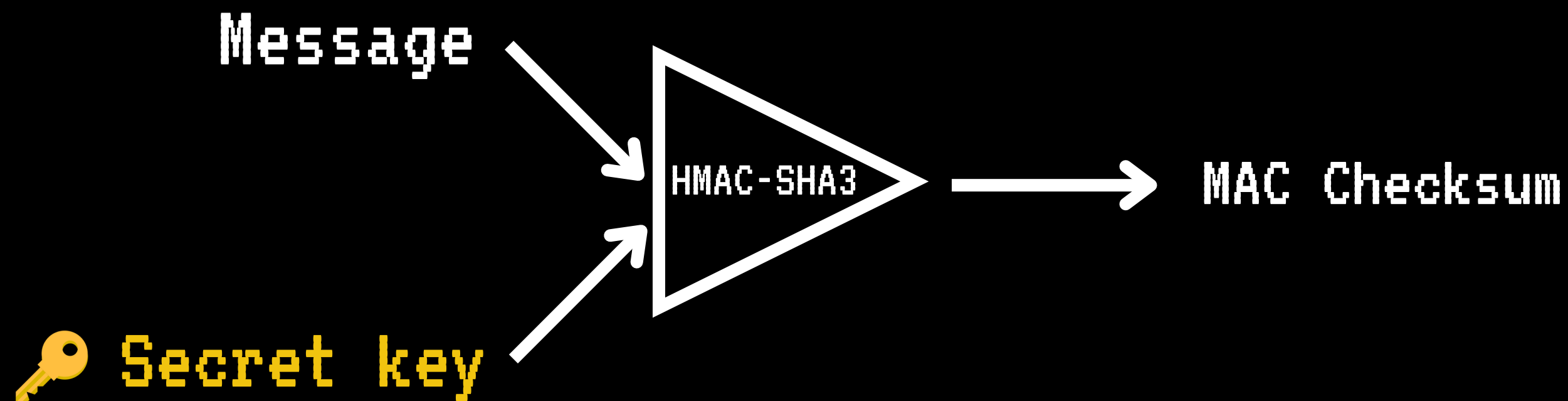
$$\text{Hash}(\text{message}_1) = \text{Hash}(\text{message}_2)$$

$$\implies \text{MAC}_{\text{key}}(\text{message}_1) = \text{MAC}_{\text{key}}(\text{message}_2)$$

Works for any hashing algorithm vulnerable to collisions (SHA1, MD5)

Hash-based MAC Impl (HMAC)

MAC by revising existing hash algorithms. Separates
key from message



- HMAC-SHA256
- HMAC-SHA3
- HMAC-DRBG
- ...

Not vuln to any of
previous attacks

HMAC Algorithm

$$\text{HMAC}(K, m) = \text{Hash}((K' \oplus \text{opad}) || \text{Hash}(K' \oplus \text{ipad} || m))$$

Where:

- Hash is the hash function
- m is the plaintext message
- K is the secret key
- K' is block-size secret key

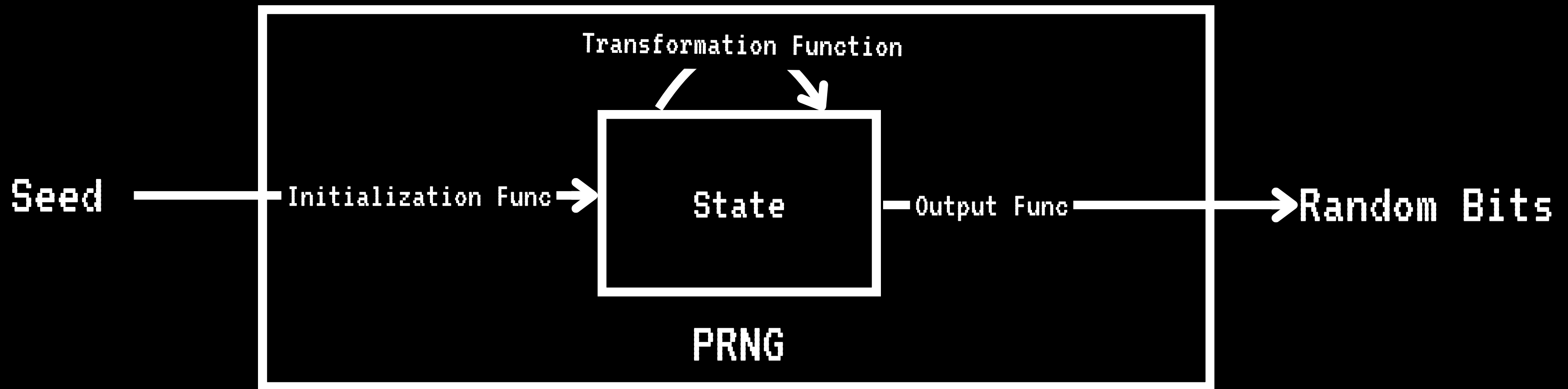
$$K' = \begin{cases} H(K) & \text{if } K \text{ is larger than block size} \\ K & \text{otherwise} \end{cases}$$

- opad is padding consisting of repeated bytes {0x5c, 0x5c, ...}
- ipad is padding consisting of repeated bytes {0x36, 0x36, ...}

Pseudo-Random Number Generators

Pseudorandom Number Generator

A deterministic algorithm that produces an arbitrary-length random value from an arbitrary given seed.

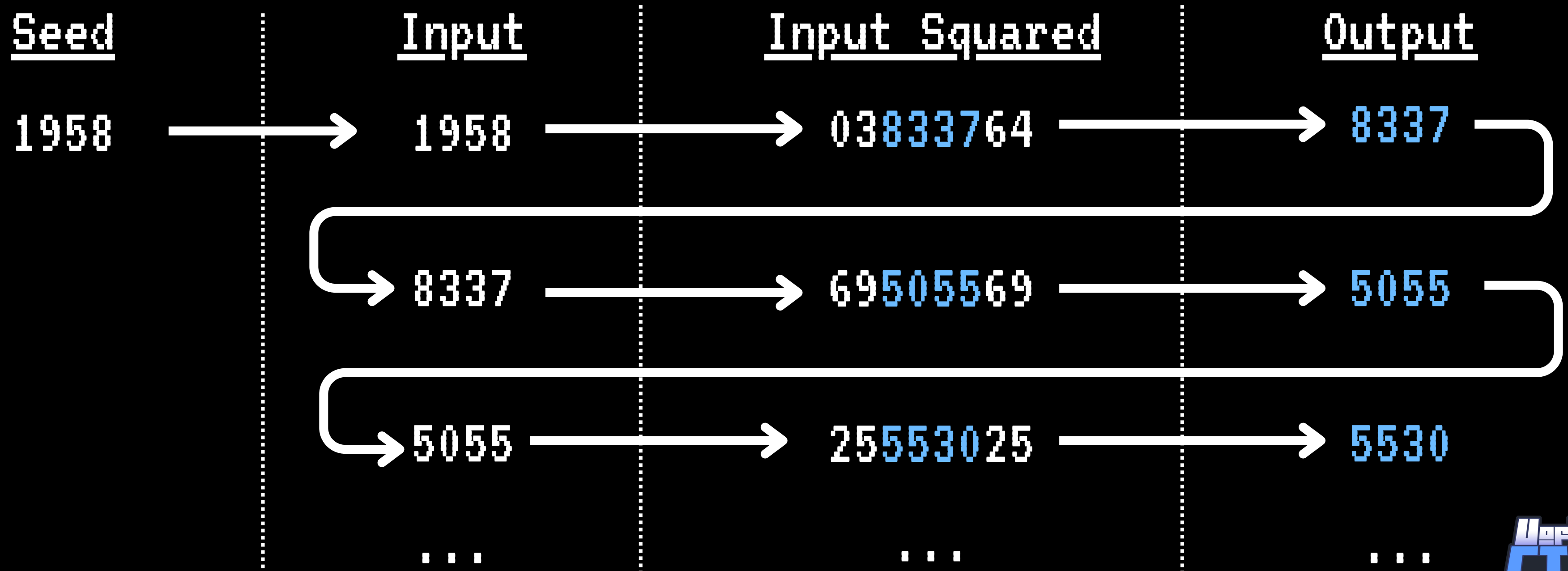


Nothing from a PRNG is truly random - finding the same seed produces the same outputs

Middle Square Algorithm

An ancient, ancient pseudo-random number generator

Take an input > 1000 , square it, get the middle 4 digits



PRNG Periods

Since PRNGs are deterministic, if you get an input that was used previously your algorithm will generate a cycle of same numbers

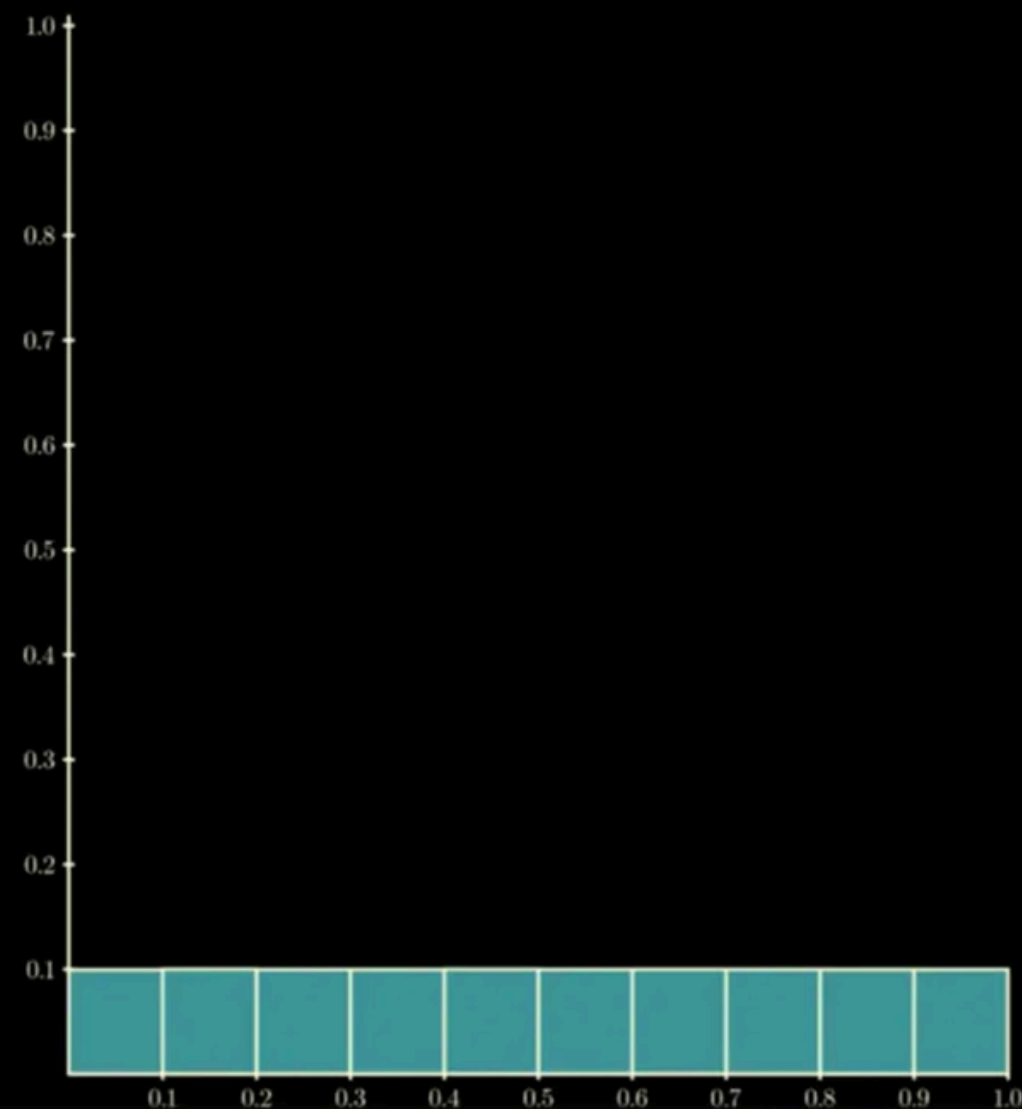
<u>Input</u>	<u>Input Squared</u>	<u>Output</u>
3600	12960000	9600
9600	92160000	1600
1600	02560000	5600
5600	31360000	3600
3600	12960000	9600
9600	92160000	1600
1600	02560000	5600
5600	31360000	3600

Uniform Distribution Test

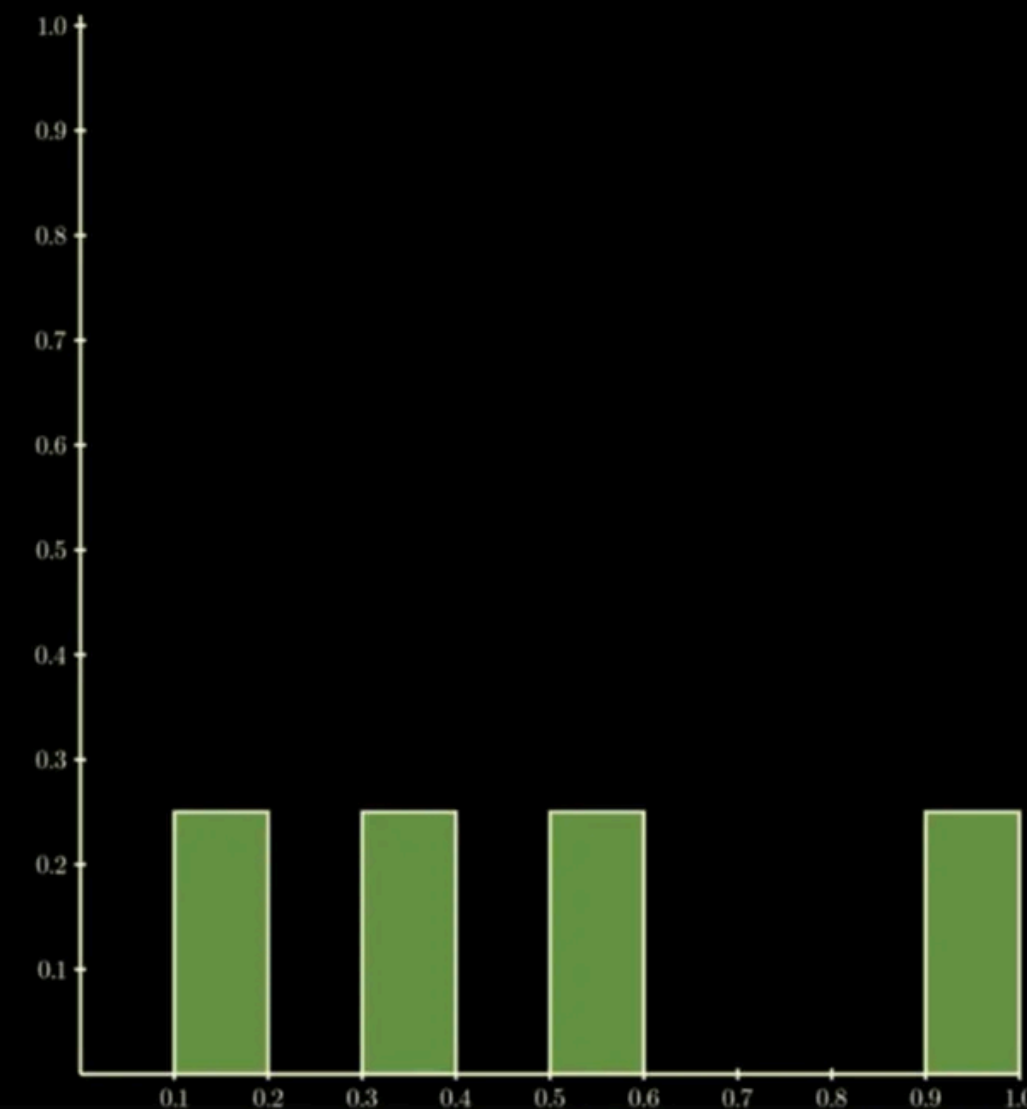
A good PRNG must have a uniform continuous distribution of results, else there will be bias to specific values

Sample size = 10,000,000

Mersenne-Twister (seed=1958)



Middle-Square (seed=1958)



Linear Congruential Generator (LCG)

A non-cryptographic secure PRNG

$$X_{n+1} = (aX_n + c) \mod m$$

Where:

- X_0 is the given seed, $0 < X_0$
- m is a given modulus constant, $0 < m$
- a is a given multiplier constant, $0 < a < m$
- c is a given increment constant, $0 \leq c < m$

LCG Example

With:

$$X_{n+1} = (1314X_n + 511) \mod 1011, X_0 = 42$$

$$X_1 = 94$$

$$X_2 = 685$$

$$X_3 = 811$$

$$X_4 = 571$$

$$X_5 = 643$$

⋮



4294685811571643...

└───┘
4 digits

You can take a substream of
this stream of random bytes
to get fixed size randomness

LCG Cracking - Recovering Parameters

With LCG defined as: $X_{n+1} = (a \cdot X_n + c) \mod m$

Recovering m:

- Define $Y_n = X_{n+1} - X_n$
- Define $u_n = |Y_{n+2} \cdot Y_n - Y_{n+1}^2|$
- $m = \text{GCD}(u_1, \dots, u_k)$ with $k \geq 10$ usually

Recovering a:

- Recover a from: $Y_i = a \cdot Y_{i-1} \mod m$

Recovering c:

- Recover c from: $X_{i+1} = a \cdot X_i + c \mod m$

DVSorter Vulnerability



A situation wherein an vulnerable LCG was used to generate voting ballots.

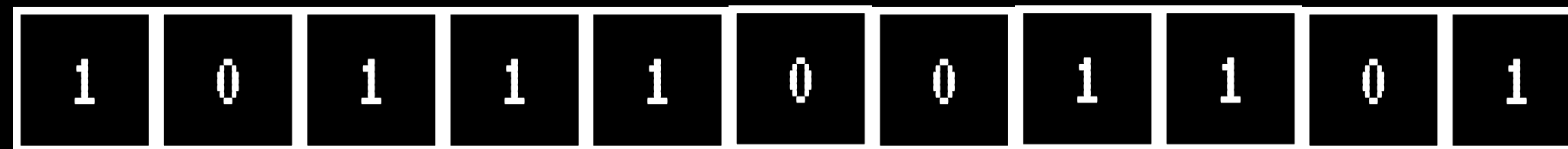
Could be used to identify voter behavior, and leak confidential voting data

XORSHIFT Generators

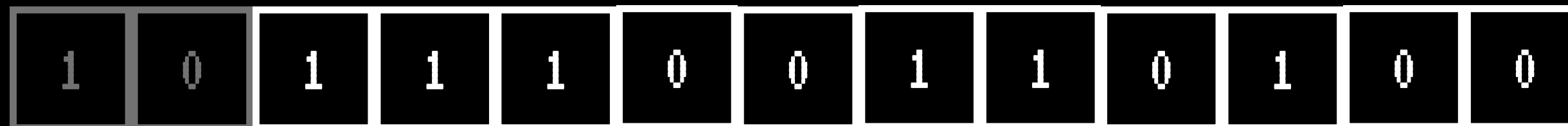
An alternative type of PRNG class that involves a combination of:

- Copying current state and **bitshifting** it
- **XOR-ing** bitshifted version(s)

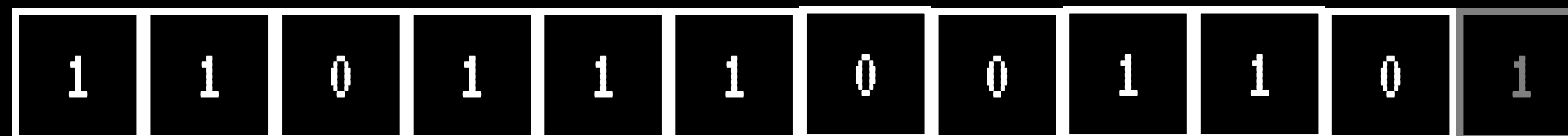
state



state $\ll_2$



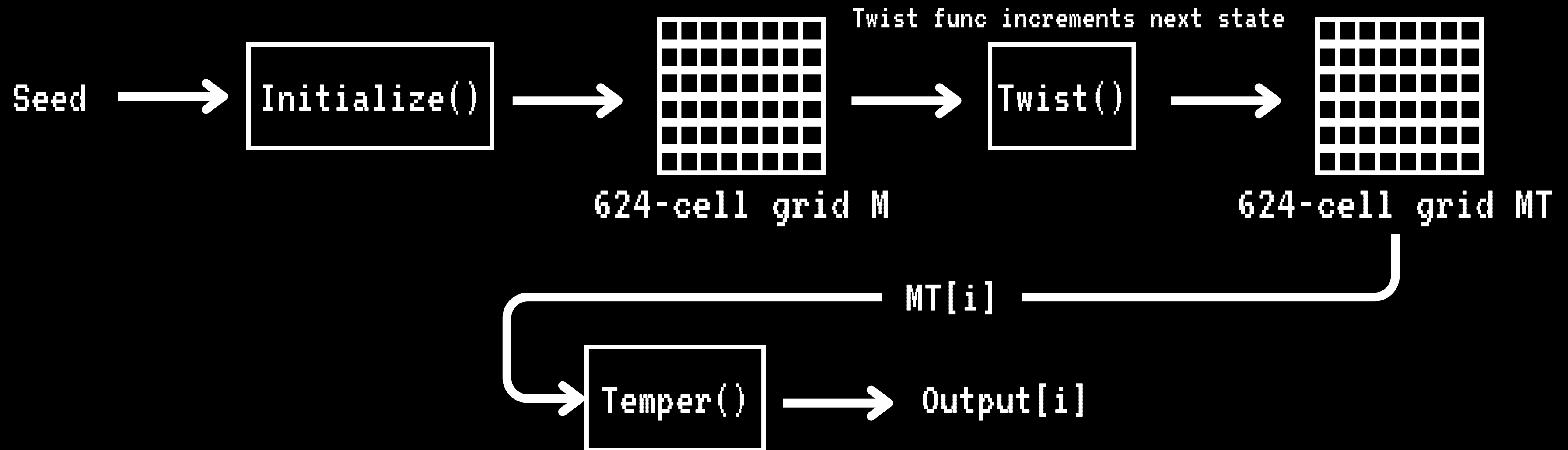
state $\gg_1$



⋮

Mersenne Twister

Most common random number generator in standard libraries of most programming languages.



Shown to be broken to seed cracking attacks

MT19937 Seed Cracking Attack

MT19937 can be broken with just 624 results to trace back the
initial seed

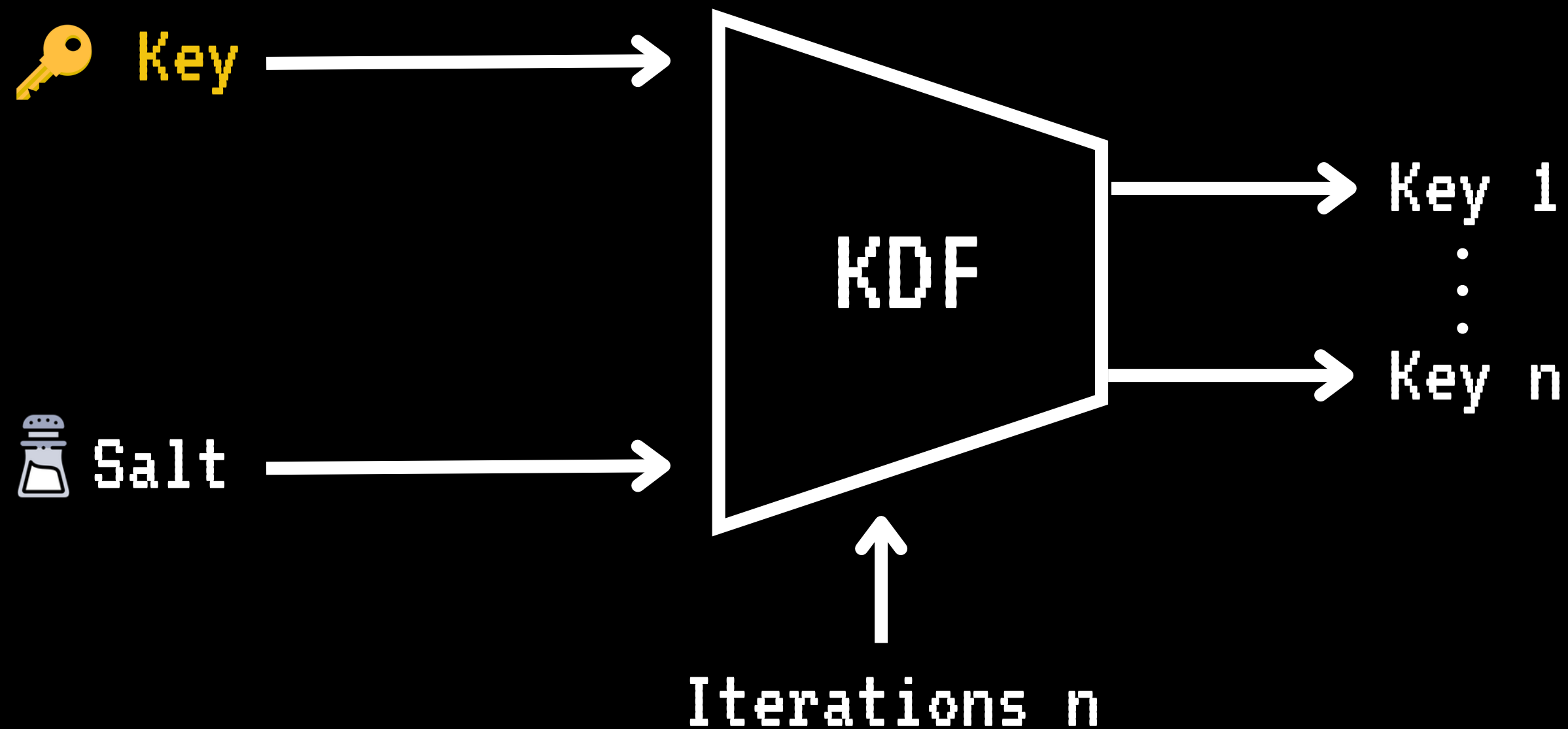
<https://pypi.org/project/randcrack/>

```
import random, time
from randcrack import RandCrack
random.seed(time.time())
rc = RandCrack()
for i in range(624):
    rc.submit(random.randrange(0, 4294967295))
print("default rand: ", random.randrange(0, 4294967295))
print("predicted rand: ", rc.predict_randrange(0, 4294967295))
```

```
default rand: 2814841327
predicted rand: 2814841327
```


Key Derivation Function (KDF)

A specific type of PRNG that is used to generate new keys from a given key.



Sidenote - True Randomness (TRNG)

You can get true randomness from:

- Driver noise (`/dev/random`)
- Lava lamps + camera (Cloudflare)
- Noises in the real world (random.org)
- Nuclear decay (HotBits)
- Coupled inverters (Intel)

Public Key Infrastructure (PKI)

Digital Signatures

Signature: An encrypted hash meant to prove one's identity.

- Everybody on the internet should be able to verify.

```
1001110000110110101010101111
11010111000100000101110000001
11000001000000110000001100111
11111001000111111001100010111
11100111110100010101010101101
11011000010111110010100101011
11100000111010010010100001011
```

Message



Digital Signature : From Alice

Done via a modification of Public-Private Key Cryptography



Public-Private Key Signatures

Using keys to prove integrity

Public Key: Shared key for validating signatures

Private Key: Private key for creating signatures

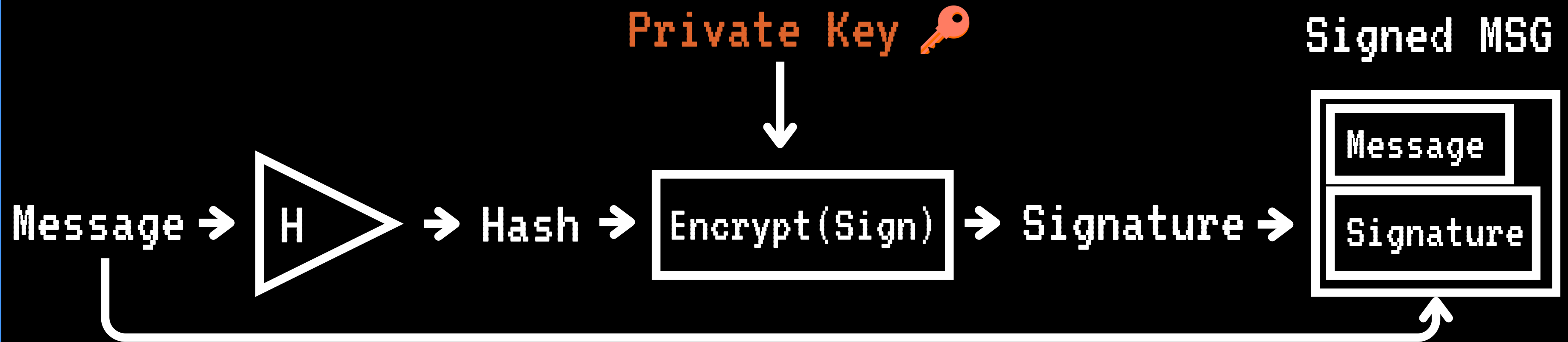
Public-Private Signing/Verifier Algorithms:

- DSA
- ECDSA
- Schnorr (ZKP)

PKI Crypto - Signing

Public Key: Shared key for validating signatures

Private Key: Private key for creating signatures

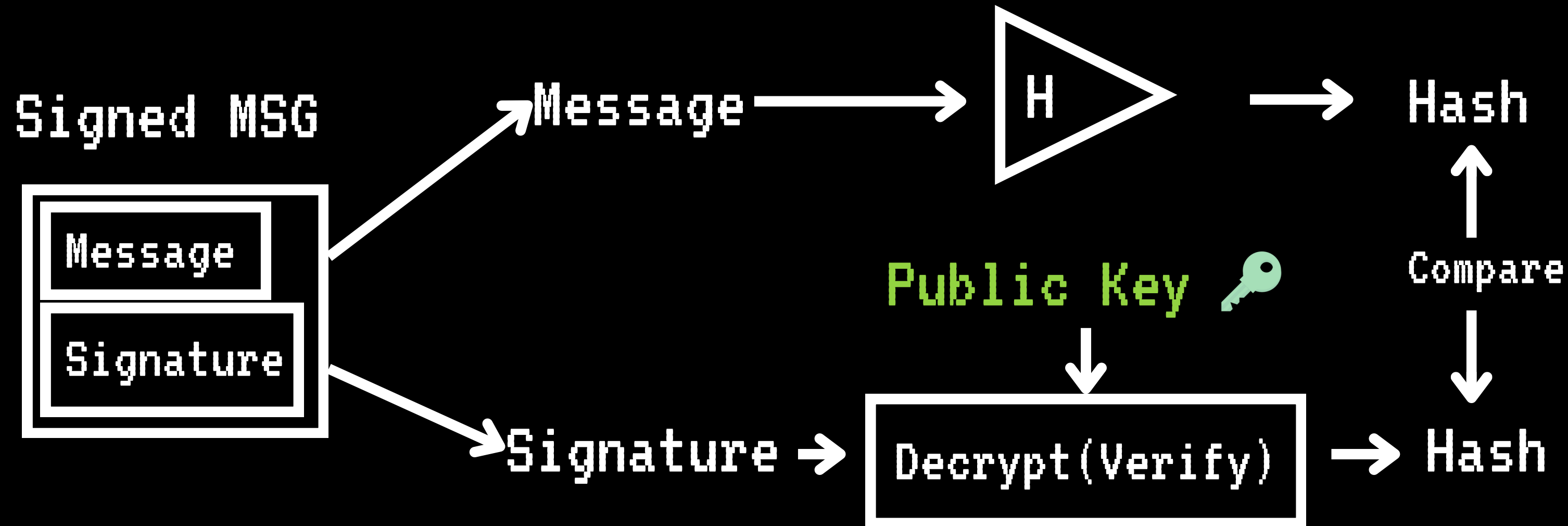


Bob signs his message and produces a signature to include with his message

PKI Crypto - Verifier

Public Key: Shared key for validating signatures

Private Key: Private key for creating signatures



Anyone verifies by comparing decrypted signature with original message hash

Nonce

A publicly shared pseudorandom number added before signing to ensure the unique signatures even for the same message.



* In british slang, nonce means s*x offender

Signatures vs MAC

MAC and Digital signatures both provide:

- Authenticity

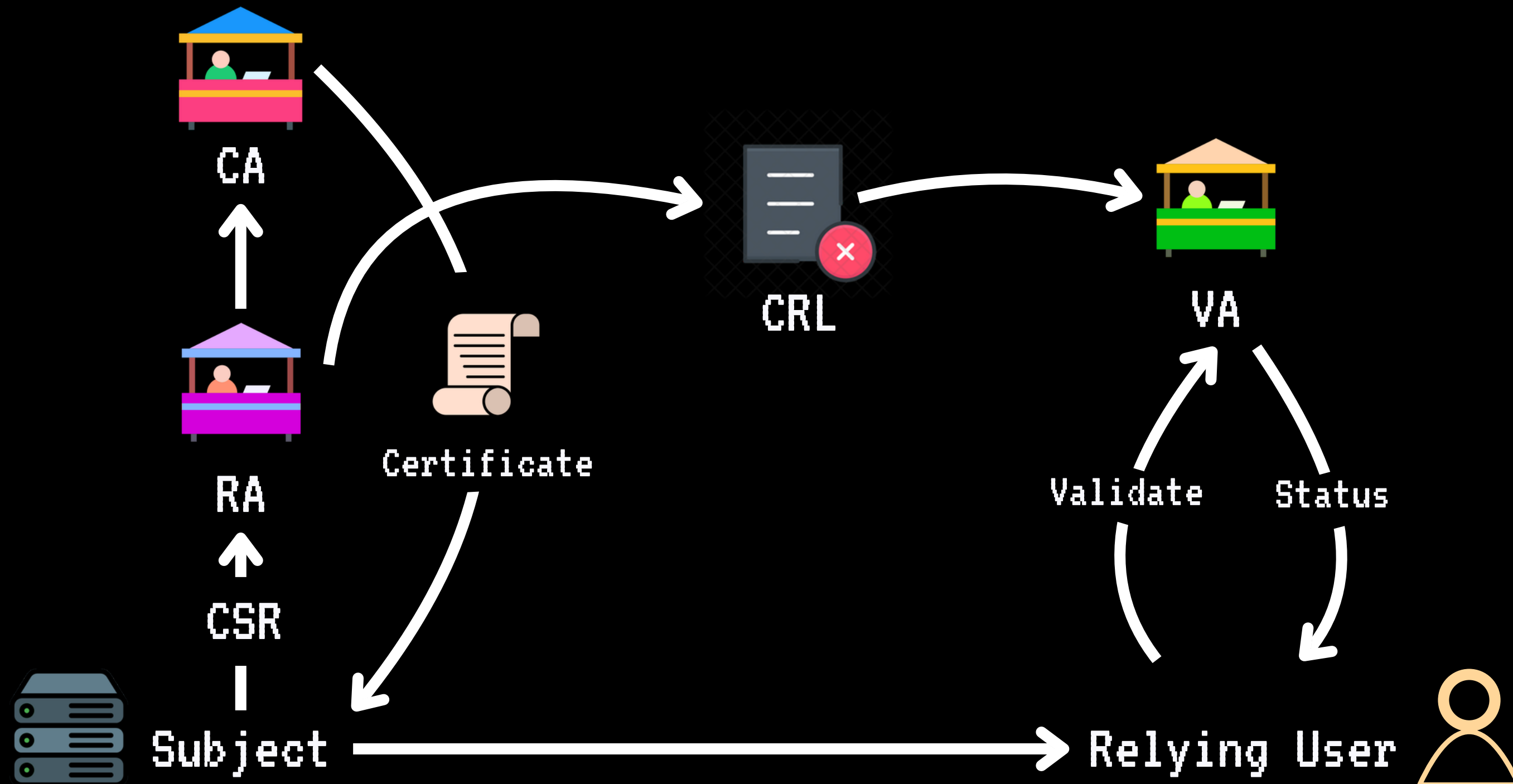
Digital signatures are different in that:

- Allows communication with strangers (no key exchange)
- They provide Non-Repudiation

Non-repudiation: Signed messages cannot be denied by sender

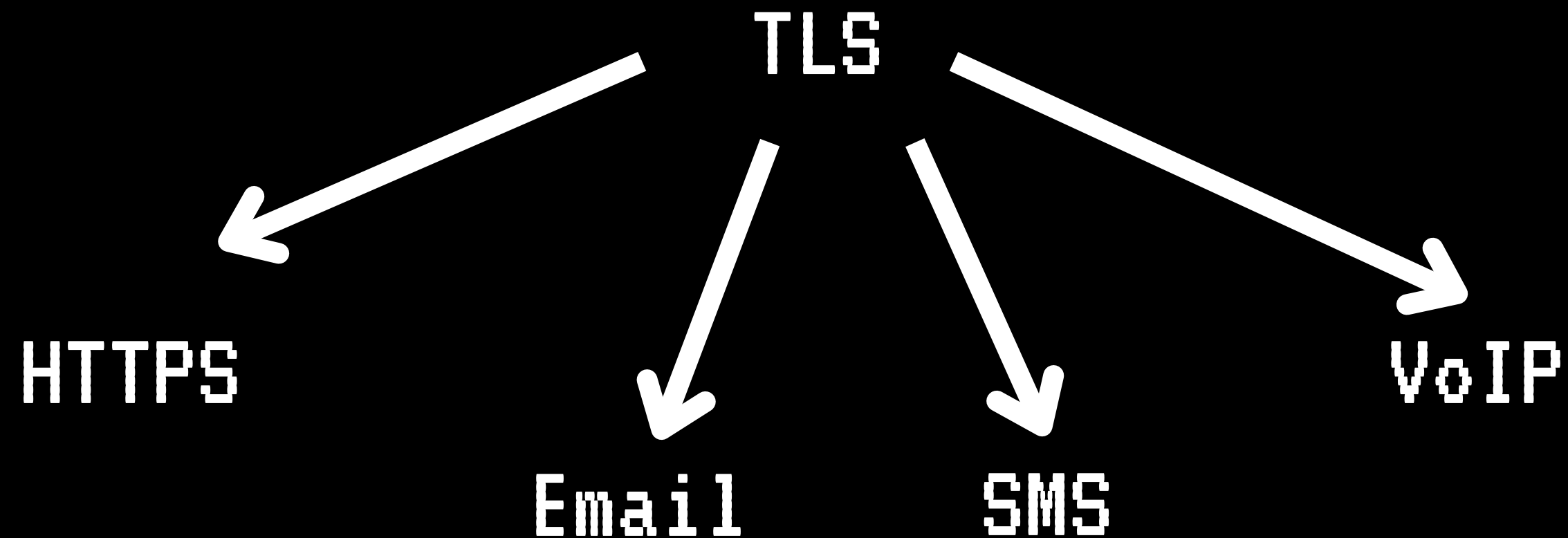
Public Key Infrastructure

A series of practices used to enforce secure communication over the internet via TLS certificates



Transport Layer Security (TLS)

TLS is the network protocol that provides confidentiality, integrity, authenticity on the internet



SSL/TLS X.509 Certificates

TLS provides security via certificates. Certificate is used to ensure a **public key** belongs to a specific entity specified by **DN**



DN: Distinguishing Name, Example:

CN=John Smith, OU=Test, O=IBM, C=GB

Certificate Authority

These are organizations responsible for creating certificates



CA

- Sectigo
- LetsEncrypt
- Verisign
- Digicert
- ...

* You can become your own CA, but you need to follow NIST guidelines + ask Firefox, Google (anything on <https://pkic.org/ltl/>) to include you on their CA list

Threat of an Evil CA

The CA is the root of trust in security. What if a CA is compromised or flat out malicious??



Evil CA

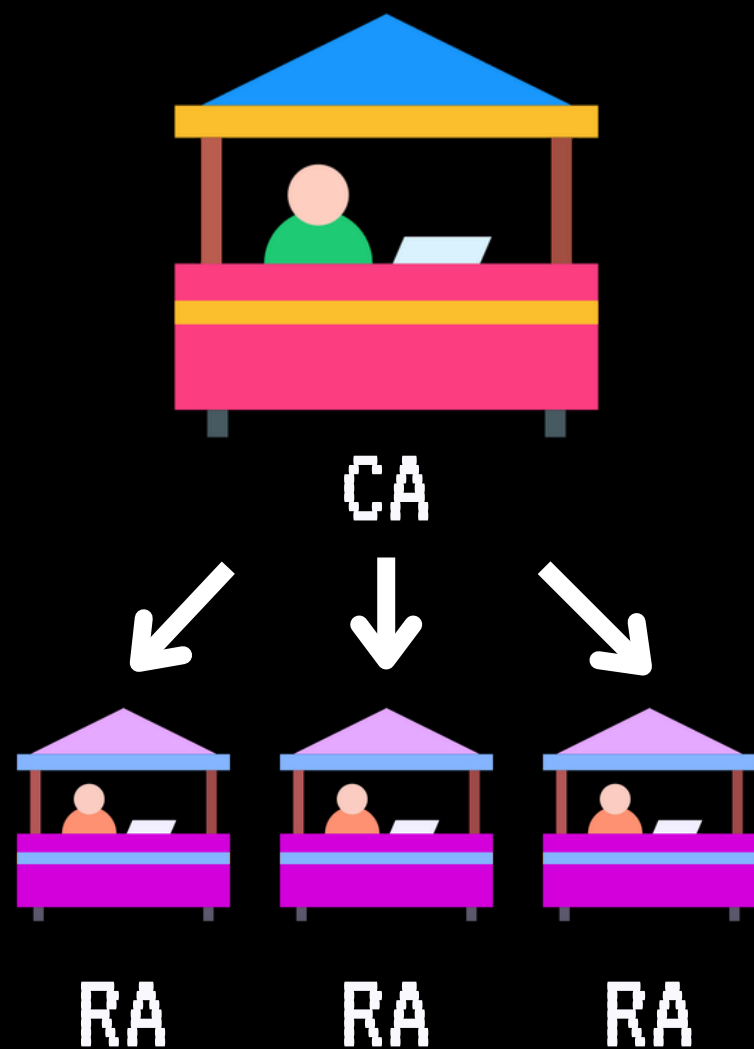
- Can falsely issue certificates for servers
- Signed malware

* Something called a Root CA is established to validate CA's because of this

Registration Authority (RA)

An intermediary between CA and PKI.

'resellers' of certificates



- Verify identities during CSR
- Revoke a certificate and add it to CRL
- Forward requests for the CA to sign

* You can apply to be an RA on the CA org's website

Certificate Signing Request (CSR)

Lets say a server wants to apply for a certificate, what information do they give the RA?



Subject



RA

Certificate Signing Request (CSR)

CSR

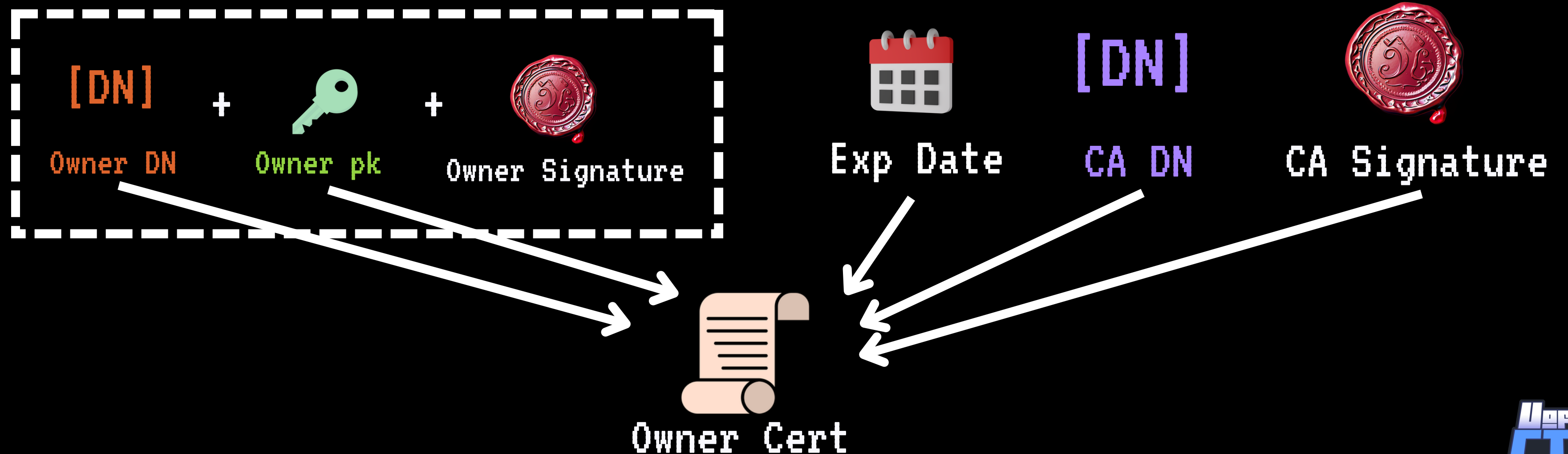


- DN for identification
- Public key
- Signature to prove access to private key

CA - Certificate Creation

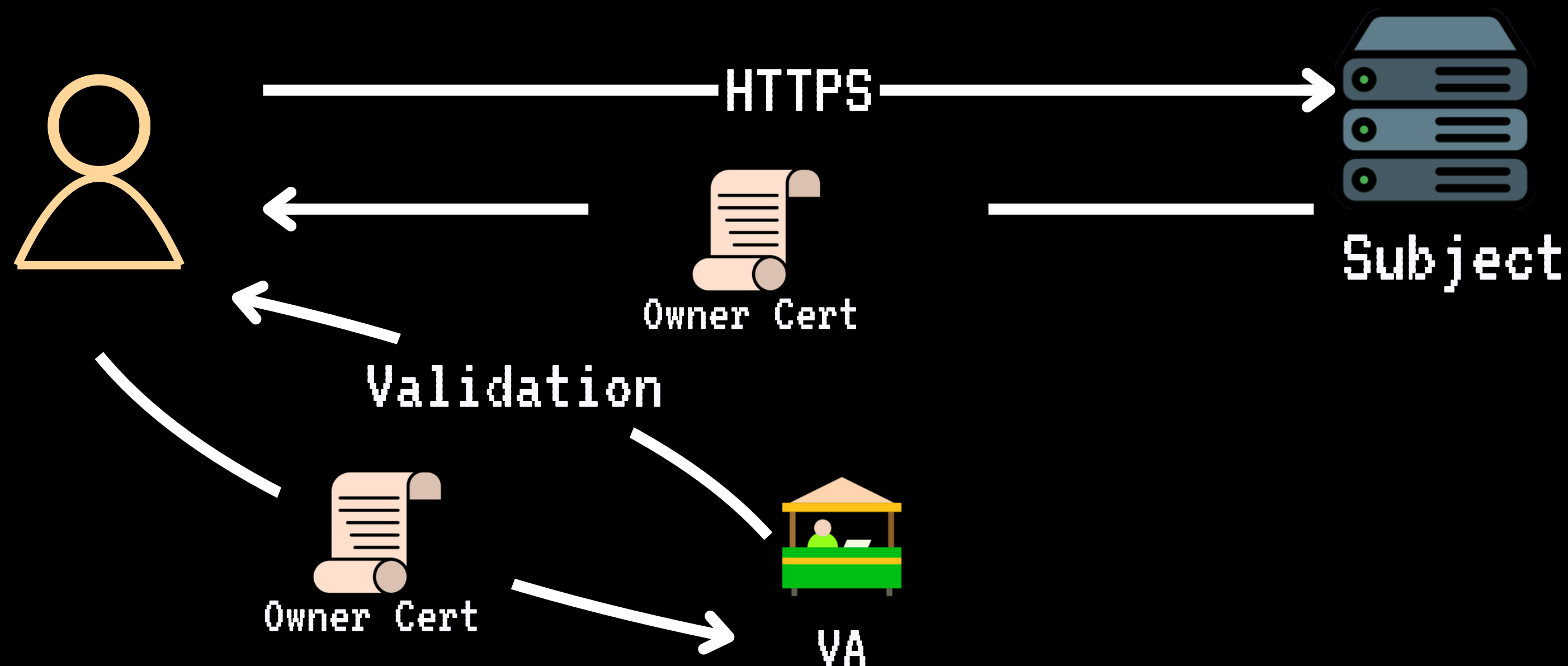
After the CSR is validated by the RA, the CA will add the time and their own signature to create the certificate

CSR



Verification Authority (VA)

Lets say the server now publishes their certificate, and a client recieves it, how does client verify? It uses a VA



Verification Authority (VA)

VA can be either (or a combination of):

- Certificate chain (you can see the chain of trust)
- Certificate revocation list (CRL)
- Online Certificate Status Protocol (OCSP)



VA

Certificate Revocation List (CRL): A list of certificates that have been expired/revoked

More Crypto If youre interested:

- Key security (salting, pepperering, lengthening, encapsulation)
- Zero Knowledge proofs
- Blockchain
- Post-quantum cryptography
- Lattice-based cryptography

Resources

Courses + Books

- https://ia601505.us.archive.org/22/items/strategic_intelligence_network/communication/encryption/introduction_to_modern_cryptography.pdf
- <https://textbook.cs161.org/crypto/>
- <https://crypto.stanford.edu/~dabo/cs255/syllabus.html>
- https://www.cs.toronto.edu/~akshayaram/CSC2426_Fall123.html
- <https://joyofcryptography.com/otp/>
- <https://sp25.cs161.org/>
- <https://pwn.college/intro-to-cybersecurity/cryptography/>
- <https://www.cryptography-textbook.com/>

Resources

Wargames:

- <https://cryptohack.org/challenges/>

PRNG:

- <https://www.youtube.com/watch?v=TF4PLUcJ05w>
- <https://diceware.blogspot.com/2012/03/as-promised-here-are-some-suggestions.html>
- <https://pit-claudel.fr/clement/blog/how-random-is-pseudo-random-testing-pseudo-random-number-generators-and-measuring-randomness/>

Resources

RSA:

- https://www.members.tripod.com/irish_ronan/rsa/attacks.html

PKI:

- <https://www.youtube.com/channel/UCPsLAM3C2BsGRv7V9h1qMtQ>
- <https://www.youtube.com/channel/UCf-SZJdgPu9E8i1qWpTSx0A>